

**Debreceni SZC Mechwart András
Gépipari és Informatikai Technikum**



Szoftverfejlesztő-és tesztelő képzés
Vizsgaremek dokumentáció

RESTAURANT MANAGEMENT SYSTEM

**Készítette: Oláh Eszter
Sulcz Kristóf
Vad Levente**

**Debrecen
2026**

Tartalomjegyzék

1.	Bevezetés	1
1.1.	A projekt bemutatása	1
1.2.	A témaválasztás indoklása	1
1.3.	Projektcélok és a vizsgaremek hatóköre	1
1.4.	A rendszer áttekintése	4
2.	Projekttervezés és csapatmunka	4
2.1.	Projektindító megállapodás és alapidokumentumok.....	4
2.2.	Fejlesztési módszertan	5
2.3.	Feladatmegosztás és kommunikáció	5
2.4.	Trello.....	6
2.5.	GitHub.....	6
3.	Követelményelemzés és rendszertervezés	7
3.1.	Funkcionális követelmények.....	7
3.2.	Nem funkcionális követelmények.....	8
3.3.	Felhasználói szerepkörök	9
3.4.	Rendszer architektúra.....	10
3.5.	REST API architektúra	11
4.	Adatmodell és adatbázis-tervezés	12
4.1.	ER diagram	12
4.1.1.	Az RMS adatmodelljének kapcsolati része	13
4.1.2.	Az RMS adatmodelljének további adattáblái	14
4.2.	Adattáblák áttekintése	14
4.3.	Adattáblák részletes bemutatása	15
4.3.1.	Felhasználók és jogosultságok.....	15
4.3.2.	Rendelések.....	15
4.3.3.	Menü és étlap.....	15
4.3.4.	Foglalások és kapcsolatfelvétel	16
4.3.5.	Nyitvatartás és rendszerbeállítások.....	16
4.4.	Bővíthetőség	16
5.	Backend megvalósítása.....	17
5.1.	A backend felépítése és az alkalmazások szerepe	17
5.2.	Az adatmodellek megvalósítása.....	18
5.3.	API réteg és végpontok	21
5.4.	Autentikáció	23
5.5.	Jogosultságkezelés	25
6.	Frontend megvalósítása	26
6.1.	Mappastruktúra	26

6.2.	Oldalak felépítése.....	28
6.2.1.	Publikus oldalak - sablonok öröklése	28
6.2.2.	Dashboard és Vendégközpont - önálló oldalak	28
6.3.	Közös felületi elemek	29
6.4.	JavaScript modulok.....	30
6.5.	Reszponzív kialakítás.....	31
6.6.	Felhasználói élmény (UX)	32
6.6.1.	Egységes megjelenés	32
6.6.2.	Azonnali visszajelzés.....	32
6.6.3.	Élő frissítés	32
6.6.4.	Kényelmi automatizmusok	32
6.6.5.	Akadálymentesség	32
6.6.6.	Szerepkör szerinti irányítás.....	32
7.	A rendszer működése.....	33
7.1.	Az alkalmazás által biztosított fő működési folyamatok	33
7.2.	A működési folyamatok bemutatása	42
8.	Tesztelés	46
9.	Összegzés.....	46

1. Bevezetés

1.1. A projekt bemutatása

A Restaurant Management System (RMS) egy éttermi folyamatok támogatására fejlesztett webalkalmazás, mely a napi működéshez és szolgáltatásokhoz kapcsolódó funkciókat biztosít vendégek és éttermi dolgozók számára egyaránt. Az RMS felhasználóbarát, letisztult felületén az ügyfelek egyszerűen tájékozódhatnak az étteremről, heti menüt rendelhetnek, asztalt foglalhatnak, vagy üzenetet is küldhetnek, miközben az adminisztrációs felület az alkalmazottak számára teszi lehetővé az étlap, a heti menü, a nyitvatartási idő szerkesztését, a rendelések, foglalások és az üzenetek kezelését.

A projektet a szoftverfejlesztő-és tesztelő képzés vizsgaremekeként egy fiktív vállalkozás, az Arany Fakanál Étterem működésére optimalizálva hoztuk létre.

1.2. A témaválasztás indoklása

Napjainkban a digitális szolgáltatások a vendéglátóipar működésének is meghatározó részévé váltak. A vendégek jelentős része online tájékozódik az éttermek kínálatáról és elérhetőségéről, ezért az online jelenlét ma már nem csupán versenyelőnyt jelent, hanem alapvető elvárásnak tekinthető. Egy étterem kiválasztásában a kínálat és az árak mellett egyre nagyobb szerepet kap a szolgáltatások egyszerű elérhetősége is. Az online asztalfoglalás, a néhány lépésben leadható rendelés vagy a gyors kapcsolatfelvételi lehetőség olyan kényelmi funkciók, amelyek pozitívan befolyásolhatják a vendégek felhasználói élményét, ezáltal hozzájárulhatnak az adott vendéglátóhely versenyképességéhez.

A munkafolyamatok digitalizációja az éttermek működésének hatékonyságát is növeli. Egy központi rendszer használata esetében, mely egyetlen felületen, naprakészen tárolja a beérkezett rendeléseket, üzeneteket és asztalfoglalásokat, átláthatóbbá és gördülékenyebbé válnak a folyamatok, csökken az adminisztrációs teher, ezáltal a hibalehetőség, valamint folyamatos az információáramlás.

A vizsgaremek témaválasztása során olyan alkalmazási területet kerestünk, amely valós üzleti igényre kínál megoldást, lehetőséget biztosít a képzés során megszerzett ismeretek gyakorlati alkalmazására, és megfelel a projektmunkával szemben támasztott követelményeknek. Egy étterem működéséhez kapcsolódó folyamatok, mint például a rendeléskezelés, asztalfoglalás, felhasználókezelés, jól elkülöníthető funkcionális egységekre bonthatók, így megfelelő alapot biztosítottak a vizsgaremek célkitűzéseinek megvalósításához.

1.3. Projektcélok és a vizsgaremek hatóköre

Elsődleges célul azt tűztük ki, hogy egy modern, esztétikus és valós üzleti értékkel bíró webes éttermi alkalmazást hozzunk létre. A projektmunka nem ügyfélmegrendelésre készült, így a személyes, vendégként szerzett tapasztalataink alapján, a termékfejlesztés logikai sorrendjét követve írtuk meg a víziót, a user story-kat és a Minimum Viable Product (MVP) követelményeket.

Projektvízió

A projekt kezdetén meghatároztuk a fejlesztés hosszú távú célját és irányát, melyet a projektvízió foglal össze.

Az RMS víziója: „Egy modern, Django alapú éttermi rendszer kialakítása, amely a manuális folyamatokat automatizált digitális munkafolyamatokká alakítja. Célunk egy olyan skálázható platform létrehozása, amely könnyen kezelhető, a vendégek számára gyors és egyszerű módot kínál az étterem szolgáltatásainak igénybevételére, az üzletvezetésnek pedig valós idejű, átlátható kontrollt biztosít a mindennapi műveletek terén.”

User story

A felhasználói igényeket user story-k formájában foglalmaztuk meg. A vendégek és az adminisztrátorok perspektívájából megírt történetek segítségével határoztuk meg a legfontosabb szoftverfunkciókat és a fejlesztés prioritásait.

RMS – vendég user story-k:

- Mint vendég, azért szeretném böngészni az étlapot a weboldalon, hogy előre tájékozódhassak a kínálatról és az árakról.
- Mint vendég, azért szeretném látni a nyitvatartási időt és az elérhetőséget, hogy pontosan tudjam, mikor tart nyitva az étterem és hol található.
- Mint vendég, azért szeretnék online asztalt foglalni, hogy a kiválasztott időpontban biztosítva legyen a helyem.
- Mint vendég, azért szeretnék felhasználói fiókot létrehozni és abba bejelentkezni, hogy heti menüt rendelhessek.
- Mint vendég, azért szeretnék a weboldalon keresztül üzenetet küldeni az étteremnek, hogy a kérdéseimet gyorsan, levelezőrendszer használata nélkül továbbíthassam.

RMS – adminisztrátor user story-k:

- Mint adminisztrátor, azért szeretném szerkeszteni az étlapot, hogy a vendégek számára naprakész információkat szolgáltatassak a kínálatunkról.
- Mint adminisztrátor, azért szeretném rögzíteni, szerkeszteni a heti menüt, hogy a vendégek online rendelhessenek a kínálatunkból.
- Mint adminisztrátor, azért szeretném látni a heti menü rendelésére vonatkozó adatokat, hogy gondoskodhassak a rendelések elkészítéséről, kiszállításáról.
- Mint adminisztrátor, azért szeretném látni az asztalfoglalásokat, hogy gondoskodhassak az asztalok előkészítéséről.

Minimum Viable Product

A user story-k alapján meghatároztuk azokat az alapvető funkciókat, melyek egy minimálisan életképes, és a vizsgáremekkel szemben támasztott elvárásoknak eleget tevő terméket eredményeznek, azaz a Minimum Viable Product (MVP) részeként megvalósított funkciókat.

Publikus felület:

- Éttermi információk megjelenítése
- Étlap megjelenítése
- Heti menü megjelenítése
- Asztalfoglalás
- Regisztráció és bejelentkezés

Vendégprofil:

- Heti menü rendelése

Adminisztrátori felület (Dashboard):

- Étlap kezelése
- Heti menü kezelése
- Rendelések kezelése
- Asztalfoglalások kezelése

Az MVP funkciók megvalósításával a rendszer már alkalmassá vált alapvető éttermi szolgáltatások biztosítására, a Dashboard pedig támogatta a legfontosabb adminisztratív feladatok ellátását.

MVP-n túli fejlesztések

Az MVP megvalósítása során további fejlesztési lehetőségeket fogalmaztunk meg, melyek az RMS funkcionalitását bővítve növelik a felhasználói élményt és a rendszer használhatóságát.

- Allergénkezelés. Létrehoztunk egy allergéneket tartalmazó adattáblát. Minden, az étlapon szereplő fogás szerkesztése, vagy létrehozása esetében lehetőség van az allergének hozzárendelésére, így az étlap már ezt az információt is tartalmazza.
- Kapcsolatfelvétel. A publikus felületen helyet kapott egy kapcsolatfelvételi modul, mely azonnali üzenetküldési lehetőséget biztosít.
- Vendégprofil bővítése. A vendégprofil esetében megvalósítottuk a profilkezelés lehetőségét, így a felhasználó módosíthatja a saját adatait, le nem zárt rendeléseit, és megtekintheti korábbi rendeléseit, asztalfoglalásait.
- Dashboard funkciók bővítése. A vendégportálon feltüntetett nyitvatartási idő módosításának lehetőségét kiszerveztük a Dashboardra, és a rendelésekre, asztalfoglalásokra vonatkozó statisztikák is helyet kaptak a felületen.

1.4. A rendszer áttekintése

Az RMS három fő logikai egységből, a publikus weboldalból, a vendégprofilból és a Dashboardból áll. A rendszer fő funkcionális egységei és azok szerepe a következő.

Publikus felület

A nyilvános weboldal bárki számára megtekinthető, és a fiktív Arany Fakanál Éttermet mutatja be. A publikus felületre látogató tájékozódhat az étteremről, megtekintheti az étlapot, a heti menüt, asztalt foglalhat, üzenetet küldhet, és információt kap az étterem nyitvatartási idejéről, elhelyezkedéséről.

Vendégprofil

A vendégprofil regisztrációt követő, bejelentkezés után elérhető felület, mely lehetőséget biztosít a vendég számára heti menü rendelés leadására, a még nem feldolgozott rendelések módosítására, a saját profil adatainak szerkesztésére, illetve az asztalfoglalások és korábbi menürendelések megtekintésére.

Dashboard

A Dashboard az éttermi dolgozók számára, adminisztrátori jogosultsággal történő belépés esetén elérhető felület. A dashboard lehetőséget biztosít az étlap és a heti menü kezelésére, a publikus weboldalon megjelenő nyitvatartási idő módosítására, a beérkezett rendelések, asztalfoglalások és üzenetek kezelésére, továbbá statisztikai adatokat is megjelenít.

A három funkcionális egység egymással együttműködve alkotja az RMS rendszerét. A publikus felület biztosítja a vendégek számára elérhető szolgáltatásokat, a vendégprofil a regisztrált felhasználók számára nyújt további funkciókat, míg a Dashboard az étterem működéséhez kapcsolódó adminisztratív feladatokat támogatja. Az egyes felületek ugyanazt a központi adatbázist használják, így a módosítások és az új adatok valamennyi érintett felületen naprakészen jelennek meg.

2. Projekttervezés és csapatmunka

2.1. Projektindító megállapodás és alapidokumentumok

A vizsgaremeket háromfős csapatban készítettük el. Az első, projektindító megbeszélésen közösen meghatároztuk az együttműködés kereteit, a fejlesztési folyamat alapelveit, valamint az alkalmazott munkamódszertant. Ezeket alapidokumentumokban rögzítettük, és egy közösen használt, felhőalapú tárhelyen helyeztük el. A fájlok a fejlesztés teljes időtartama alatt referenciaként szolgáltak.

RMS Project Charter

A dokumentum tartalmazza a projekt célját, a fejlesztési szemléletet, a csapat együttműködésének szabályait, a kommunikáció rendjét, valamint a projektmenedzsmenttel kapcsolatos alapelveket. Emellett a Project Charterben határoztuk meg a projektvíziót, a user story-kat és az MVP funkcióit is.

Project Tracker

A dokumentum célja, hogy a projekt kidolgozásának, az RMS fejlesztésének státuszát naprakészen dokumentáljuk. A tracker folyamatosan frissült a fejlesztés előrehaladtával, ebben rögzítettük a

következő iterációra tervezett vállalásokat felelősökkel, illetve azon fontos kitételeket és részleteket, melyekben a csapat megegyezett.

Brand Guide

A Brand Guide az RMS megjelenésével kapcsolatos arculati irányelveket foglalja össze. Alkalmazása garantálja az egységes és harmonizáló dizájnt minden felület esetében. A dokumentum színpalettákat, betűtípusokat, UI elemeket, stílusjegyeket tartalmaz, melyeket a weboldal kialakítása esetén követni szükséges.

Az alapidokumentumok a fejlesztési folyamat szervezését, a csapat együttműködését és az egységes megvalósítást támogatják.

2.2. Fejlesztési módszertan

A vizsgaremek fejlesztése során az Agile szemlélet alkalmazását tartottuk ideálisnak, annak rugalmassága, átláthatósága és iteratív jellege miatt. A projektmunka kidolgozása nem egy előre, részleteiben rögzített projektterv szerint történt, hanem a fejlesztés előrehaladásával párhuzamosan alakult. A rendszer funkcióit fokozatosan valósítottuk meg, az elkészült részeket közösen értékeltük és teszteltük, majd a tapasztalatok alapján szükség esetén továbbfejlesztettük vagy módosítottuk. Az újonnan felmerülő ötletek, a tesztelés eredményei és a csapat közös értékelése alapján a termék fejlesztéséhez szükséges feladatlista, vagyis a product backlog is változott, így a rendszer funkcionalitása iteratív módon bővült.

A munkafolyamat leginkább a Kanban módszertanhoz hasonlított, mivel a fejlesztés egy folyamatosan priorizált backlog alapján zajlott, a feladatok állapotát pedig Kanban táblán követtük nyomon. Jellemzően hetente, ugyanabban az időpontban egyeztettünk, ám szükség szerint akár gyakrabban vagy néhány nap csúszással időzítettük a megbeszéléseket annak érdekében, hogy a munkafolyamat folytonossága biztosított legyen. A backlog elemeit a nagyobb fejlesztési feladatok felbontásával határoztuk meg úgy, hogy azok egy hét alatt is elvégezhető, ám önállóan is értelmezhető és értéket adó részfeladatok legyenek. A heti megbeszéléseink során áttekintettük az előző héten elvégzett fejlesztéseket, alkalmanként további fejlesztési lehetőségeket, igényeket határoztunk meg, illetve megállapodtunk a következő ciklusra időzített teendőkben, prioritásokban. Ezek alapján minden csapattag a rendelkezésére álló kapacitásának megfelelő mennyiségű feladatot vállalt a következő fejlesztési időszakra.

2.3. Feladatmegosztás és kommunikáció

A vizsgaremek létrehozása során a közös tervezésre és egyeztetésre épülő, majd párhuzamos fejlesztéssel megvalósított munkaszervezést alkalmaztuk. Az átláthatóságot a rendszeres megbeszélések és a közösen karbantartott Product Backlog biztosította. A szakmai egyeztetéseket Discord hangcsatornán tartottuk, míg a két megbeszélés közötti operatív kommunikáció Messenger csoportban zajlott. A projekthez kapcsolódó dokumentumokat, terveket és egyéb állományokat Google Drive mappában tároltuk, így azok minden csapattag számára folyamatosan elérhetők voltak. Mivel a backlog elemeit úgy definiáltuk, hogy a feladatok között minimális volt a függőség, nem okozott gondot a szimultán munkavégzés.

A projektindító megbeszélésen megállapodtunk a projektcélokban, majd önállóan gyűjtöttünk ötleteket az UI koncepcióhoz. A tervek alapján közösen határoztuk meg a végleges felhasználói felületet, a Brand Guide-ot, valamint a weboldal funkcionális és szerkezeti koncepcióját. Ezt követően a fejlesztés párhuzamosan zajlott, csapaton belül elkülönülő szerepköröket nem

határoztunk meg. A feladatokat érdeklődés, kompetencia és kapacitás alapján osztottuk fel egymás között, mindhárman önállóan dolgoztunk a vállalt részfeladatokon, ugyanakkor a rendszeres megbeszélések során folyamatosan egyeztettük a fejlesztési irányokat. A frontend és a backend fejlesztése párhuzamosan zajlott, majd ezt követte az egyes komponensek integrációja. Az elkészült funkciókat először egyénileg ellenőriztük és teszteltük, majd a teljes alkalmazást közösen is validáltuk. A projekt dokumentációjának elkészítését szintén felosztottuk egymás között. Minden csapattag elsősorban az általa fejlesztett funkciók és komponensek dokumentációjának elkészítéséért felelt, melyek a vizsgaremek dolgozat összeállításához szolgáltak alapul.

2.4. Trello

A projekt előrehaladásának nyomon követésére a Trello alkalmazást használtuk. A Trello lehetőséget biztosított arra, hogy a product backlog elemeit egy közösen használt felületen kezeljük, a feladatokhoz tartozó specifikációkat megadjuk, a backlog elemekhez felelősöket rendeljünk, és a készültségi állapotokat is naprakészen figyelemmel tudjuk kísérni. A virtuális tábla támogatta a Kanban módszer szerinti működésünket. A feladatok státuszának követésére listákat hoztunk létre, melyek a következők voltak:

- Product Backlog, amely tartalmazta a projekt fejlesztéséhez szükséges, még el nem kezdett feladatokat,
- To Do, az aktuális fejlesztési ciklus vállalásai,
- In Progress, a folyamatban lévő fejlesztések listája,
- On Hold, a külső függőség vagy elakadás miatt szünetelő feladatok listája,
- In Review/Testing, a már elkészült, de közösen értékelendő, vagy tesztelés alatt álló feladatok listája,
- Done, az elkészült és elfogadott kártyák listája.

A feladatkártyákat színekkel címkézve jelöltük meg, melyek a különböző funkcionális területek elkülönítését szolgálták.

A Trello használata hozzájárult a fejlesztési folyamat átláthatóságához, ezzel segítve a csapatmunka hatékonyságát.

2.5. GitHub

A projekt forráskódjának kezelésére GitHub repositoryt használtunk, amely biztosította a közös fejlesztéshez szükséges verziókövetést, az elkészült módosítások visszakövethetőségét és támogatta a párhuzamos munkavégzést. A repository lehetővé tette a különböző fejlesztési ágakon végzett munka biztonságos elkülönítését egy egységes munkakörnyezeten belül.

A fejlesztési folyamat során többszintű branch-stratégiát alkalmaztunk.

Branch	Szerepe
Master	A repository elsődleges ága, amely a projekt stabil, működő verzióját tartalmazza.
Beta	A fő ágból leágazó beta az integrációs feladatok elvégzésére szolgált, ahová a feature brancheken elkészült fejlesztések kerültek beolvasztásra.
Feature	A beta ágból a különböző funkciók fejlesztéséhez külön feature brancheket hoztunk létre. Ezeken az ágakon a fejlesztések egymástól függetlenül készültek el anélkül, hogy közvetlenül befolyásolták volna a projekt aktuális, stabil állapotát.

Táblázat: A fejlesztésben használt Git-ágak szerepe

Az új funkciók befejezése és ellenőrzése után a módosításokat először a beta ágba integráltuk, majd a sikeres összefűsölést és tesztelést követően kerültek át a master ágba. Ez a munkafolyamat csökkentette az integráció során fellépő hibák kockázatát, valamint biztosította a fejlesztések fokozatos és ellenőrzött beépítését.

A verziókövetés átláthatóságát a tudatos commit-kezelés is támogatta. A commitok rövid, egyértelmű leírásokat tartalmaztak, amelyek megkönnyítették az elvégzett módosítások nyomon követését, valamint szükség esetén a korábbi állapotok visszakeresését. A GitHub alkalmazása jelentősen hozzájárult a párhuzamos fejlesztés biztonságos megvalósításához, a forráskód egységének megőrzéséhez és a csapat hatékony együttműködéséhez.

3. Követelményelemzés és rendszertervezés

3.1. Funkcionális követelmények

A funkcionális követelmények azt írják le, mit tudjon az alkalmazás. Az alábbiakban három csoportra bontjuk a követelményeket: a vendég és a nyilvános felület, az admin dashboard, végül azok a funkciók, amelyek mindkét oldalon közzé esnek.

Vendég és nyilvános felület

A nyilvános oldalakat bejelentkezés nélkül is lehet böngészni. Regisztráció és belépés után elérhető a kosárból rendelés és a vendégközpont; az asztalfoglalás belépés nélkül is működik. Ezen a felületen a rendszernek a következőket kell tudnia:

- Regisztráció, bejelentkezés és kijelentkezés.
- A saját fiókadatok kezelése a vendégközpontban (név, telefon, cím).
- Az állandó étlap megtekintése (kategóriák, árak, allergének).
- A heti menü megtekintése és kosárba helyezés: naponta két választás van (A és B menü), napokra bontva.
- Rendelés leadása kiszállítási címmel — csak bejelentkezés után.
- Asztalfoglalás (dátum, időpont, létszám, alkalom) — bejelentkezés nélkül is.

- Bejelentkezett felhasználónál a foglaló úrlapon a név, e-mail és telefon előtöltése.
- Kapcsolatfelvételi üzenet küldése.
- A nyitvatartás megjelenítése (heti sablon és kivételek, például ünnepnap).
- A vendégközpontban a saját rendelések és foglalások listája; amíg egy tétel „Új” vagy egy foglalás függőben van, módosítás vagy lemondás.
- Adatvédelmi tájékoztató és beleegyezés a releváns úrlapokon.

Admin (dashboard)

Az admin felület az étterem napi munkáját támogatja. Itt a rendszernek a következőket kell biztosítania:

- Áttekintő kezdőlap a fontos számokkal (például a mai rendelések és foglalások) és a teendőkkel, figyelmeztetésekkel.
- A rendelések kezelése: lista, státuszváltás, szerkesztés, törlés.
- A foglalások kezelése: lista, státuszváltás, szerkesztés, törlés.
- Az étlap, a kategóriák és a heti menü szerkesztése.
- A beérkezett kapcsolatüzenetek olvasása és archiválása.
- A nyitvatartás beállítása, valamint az SLA időkorlátok megadása (az SLA azt mondja meg, mennyi időn belül kellene reagálni egy rendelésre vagy foglalásra).

Közös funkciók

Vannak olyan követelmények, amelyek nem csak a vendéghez vagy csak az adminhoz kötődnek, hanem mindkét felületen számítanak. Ezek közül a legfontosabb:

- Élő frissítés — Ha egy másik ablakban megváltozik valami, a lista magától frissül a revision / live-sync mechanizmuson keresztül.

3.2. Nem funkcionális követelmények

A nem funkcionális követelmények nem azt mondják meg, mit csinál a rendszer, hanem azt, milyen minőségben működjön: hogyan jelenjen meg, mennyire legyen biztonságos, mennyire legyen karbantartható és bővíthető. Az alábbiakban négy csoportra bontjuk ezeket.

Megjelenés és használhatóság

A felületnek érthetőnek és kényelmesen használhatónak kell lennie különböző eszközökön. Ehhez a következőket várjuk el:

- Reszponzív megjelenés: telefonon és nagyobb képernyőn is jól olvasható és kezelhető legyen.
- Érthető visszajelzések siker és hiba esetén — nálunk ezt rövid felugró üzenetek (toast) jelzik.

Biztonság

A rendszernek védenie kell a felhasználói adatokat és az admin műveleteket. Fontos elvárások:

- Biztonságos bejelentkezés: a munkamenetet session cookie tartja fenn, a módosító kéréseknél pedig CSRF-védelem is legyen (hogy idegen oldal ne tudjon műveletet indítani a nevében).

- A jelszavak ne olvasható szöveggént tárolódjanak az adatbázisban — a Django hash-elit ők.
- Szerepkörök szerinti elválasztás: a vendég ne lássa az admin listákat, és ne férjen hozzá más vendég adataihoz.

Működés és teljesítmény

A mindennapi használat során a felületnek gyorsnak és gördülékenynek kell maradnia. Ehhez:

- Frissítéskor ne mindig az egész oldal töltsdön újra — elég legyen a JSON válasz és a JavaScript frissítése.
- Az élő frissítés (revision számláló) elfogadható válaszidővel működjön, felesleges teljes oldal-újrátöltés nélkül.

Karbantarthatóság és bővíthetőség

A kódnak később is átláthatónak kell maradnia, és új funkciókat ésszerű erőfeszítéssel lehessen hozzáadni. Ezért a következőket tűztük ki:

- REST API: a böngésző HTTP-n keresztül /api/... címeket hív, és JSON választ kap; közvetlenül az adatbázishoz nem nyúl. Így a frontend és a backend szétválasztható.
- Átlátható felépítés: a backend külön appokban van (például orders, menu), a frontend külön JS/CSS fájlokban (dashboard, vendégközpont). Így könnyebb megtalálni, hol mi van.
- Bővíthetőség: új funkció (például e-mail értesítés) hasonló mintával hozzáadható legyen (modell + API + JavaScript), anélkül hogy mindent újra kellene írni.
- Fixtures és verziókezelés: a mintaadatok fixtures fájlokban legyenek; a db.sqlite3 ne kerüljön a gitbe. Így mindenki ugyanazt a demót tudja betölteni, és a helyi adatbázis nem megy fel a GitHubra.

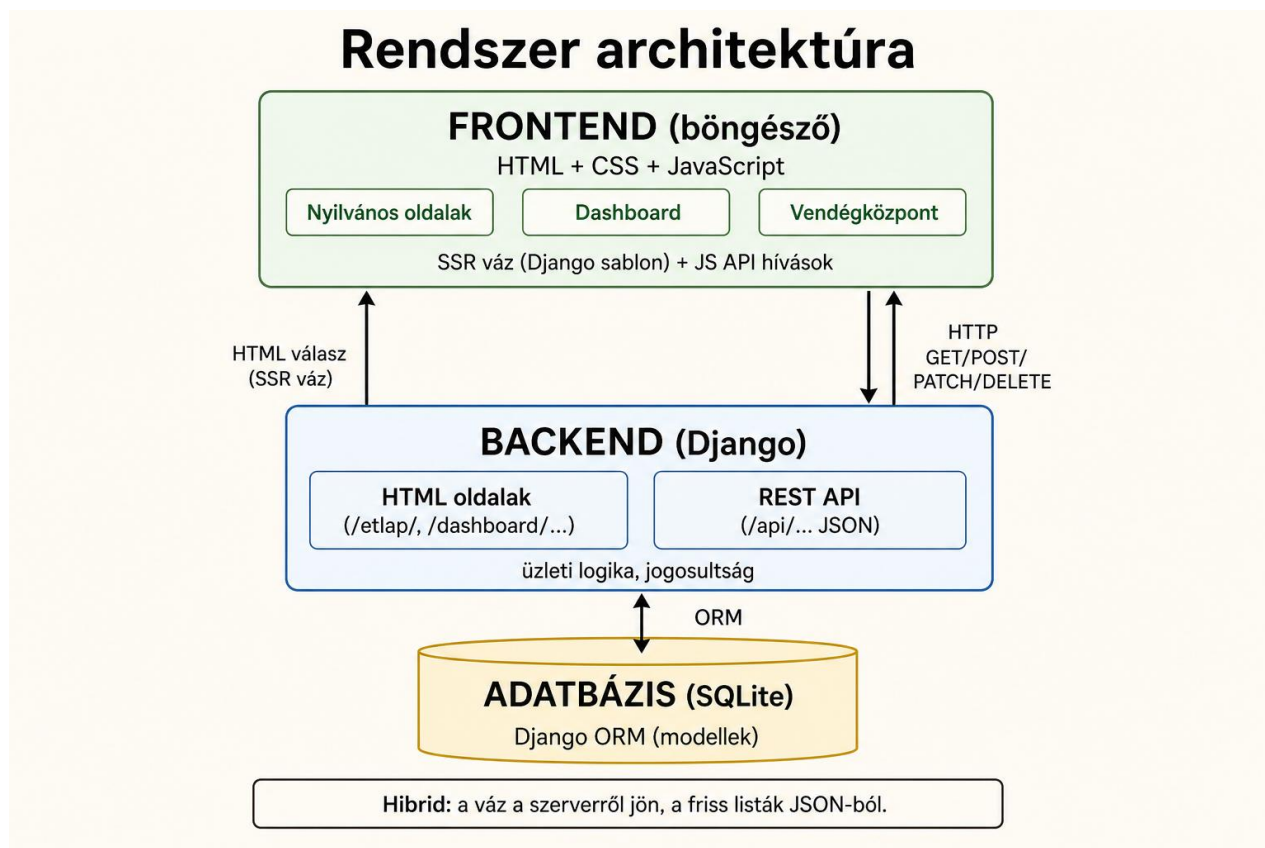
3.3. Felhasználói szerepkörök

A rendszer különböző típusú felhasználókkal dolgozik: van, aki csak böngész, van, aki bejelentkezik vendégként, és van, aki adminisztrál. A bejelentkezett felhasználó szerepkörét a UserProfile.role mező tárolja (ez az adatbázisban a felhasználóhoz tartozó profil egyik mezője). Emellett bejelentkezés nélkül is használható a nyilvános felület. Az alábbi szerepköröket különböztetjük meg:

- Vendég (bejelentkezés nélkül): megnézheti az étlapot és a heti menüt, asztalfoglalást és kapcsolatüzenetet küldhet. Rendelés leadásához viszont be kell jelentkeznie.
- Regisztrált vendég (customer): a fentiek mellett be tud jelentkezni, kosárból rendelhet, és a vendégközpontban kezeli a saját rendeléseit, foglalásait és fiókadatát.
- Admin (admin): a dashboardon kezeli a rendeléseket, foglalásokat, menüt, üzeneteket, nyitvatartást és az SLA beállításokat — az összes releváns rekordot látja és módosíthatja.
- Alkalmazott (employee): a modellben már létezik ez a szerepkör, de külön felület jelenleg nincs hozzá (ez továbbfejlesztési terület).

3.4. Rendszer architektúra

A rendszer architektúra azt írja le, hogyan épül fel az alkalmazás nagyobb darabokból. Három réteg dolgozik együtt —frontend (böngésző), backend (Django) és adatbázis. Ezt mutatja az alábbi ábra.



Ábra: Rendszer architektúra — frontend, backend, adatbázis

Frontend (böngésző)

Ez az, amit a felhasználó lát és kezel: HTML, CSS és JavaScript. Három fő felület van:

- Nyilvános oldalak — főoldal, étlap, asztalfoglalás.
- Admin dashboard — a /dashboard/ címen.
- Vendégközpont — a /guest-portal/ címen.

Hogyan készül az oldal (hibrid modell)?

Az oldalak nem egyetlen módon készülnek: a szerver és a böngésző megosztja a munkát. Konkrétan:

- A Django a sablonokból összerakja az oldal vázát (navbar, layout, űrlapok). Ezt SSR-nek (Server-Side Rendering) hívják: a szerver készíti el a HTML keretet, és úgy küldi a böngészőnek.
- A listák és táblázatok tartalmát a JavaScript kéri le az API-ról (JSON formátumban), és utána rajzolja ki.

Ez a hibrid modell: a váz a szerverről jön, a friss listákat a JavaScript tölti be. Tehát nem tiszta SSR (nem minden adat van eleve a HTML-ben), és nem is klasszikus SPA (nem egy teljesen üres alkalmazást töltesz be, ami mindent maga épít). A kettő együtt dolgozik.

Backend (Django)

A backend kétféle kérést szolgál ki: HTML oldalakat (például /etlap/, /dashboard/) és REST API-t (/api/..., JSON válasz). Itt fut az üzleti logika: ki mit láthat, érvényes-e a foglalás, menthető-e a rendelés. Az adatbázist a backend ORM-mel kezeli.

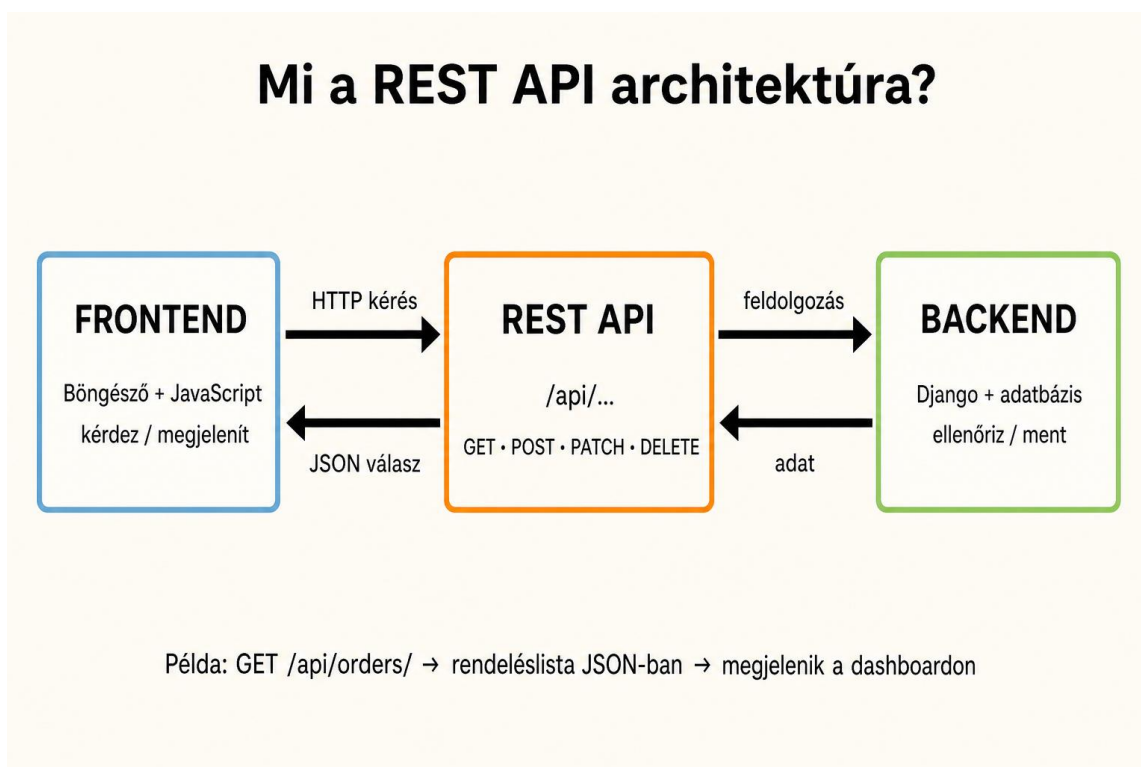
Adatbázis

Az adatokat SQLite adatbázisban tároljuk: ez egy helyi fájl, nem külön adatbázis-szerver. Ide kerülnek a felhasználók, a menü, a rendelések, a foglalások, az üzenetek és a nyitvatartás. A kódban Django modellekkel dolgozunk — ezt hívják ORM-nek (Object-Relational Mapping): Python objektumokkal írunk és olvasunk (például rendelés, foglalás), a Django pedig ebből kezeli az adatbázist; nem kell minden listához kézzel SQL-t írni. A böngésző közvetlenül nem éri el az adatbázist.

Példa a három réteg együttműködésére: az admin megnyitja a rendeléseket → betöltődik a /dashboard/ HTML → a JavaScript GET /api/orders/ kérést küld → a Django ellenőrzi a jogosultságot, ORM-mel lekéri az adatokat → JSON jön vissza → a táblázat kirajzolódik.

3.5. REST API architektúra

A REST API architektúra azt írja le, hogyan beszél a böngésző a szerverrel. A kapcsolat HTTP kéréseken és JSON válaszokon keresztül megy, jól meghatározható /api/... végpontokon. A frontend közvetlenül nem nyúl az adatbázishoz — mindig a backend API-n keresztül kér vagy küld adatot. Ezt a kapcsolatot mutatja az alábbi ábra.



Ábra: REST API — a frontend és a backend közötti kapcsolat

Gyakorlatban ez így néz ki: a dashboard megnyitásakor a JavaScript elkéri a rendeléseket (GET /api/orders/). A szerver ellenőrzi a jogosultságot, összerakja a listát, és JSON-ban visszaküldi. A

böngésző ebből rajzolja ki a táblázatot. Mentésnél ugyanez fordítva történik: a JavaScript elküldi az adatot, a server ellenőrzi és ment.

Azt, hogy mit szeretnénk csinálni az adattal, a HTTP metódus mondja meg. A fő műveletek:

- GET — olvasás: lista vagy egyetlen rekord lekérése.
- POST — új rekord létrehozása (például új rendelés vagy foglalás).
- PATCH — részleges módosítás (például státuszváltás).
- DELETE — törlés.

A REST elvek így jelennek meg: az erőforrásoknak saját URL-jük van (/api/orders/, /api/reservations/ stb.), az adatsere JSON formátumban történik, a műveletet pedig a HTTP metódus jelzi. Így a frontend és a backend világosan szétválik, és mindkét oldal ugyanazokat a végpontokat tudja használni.

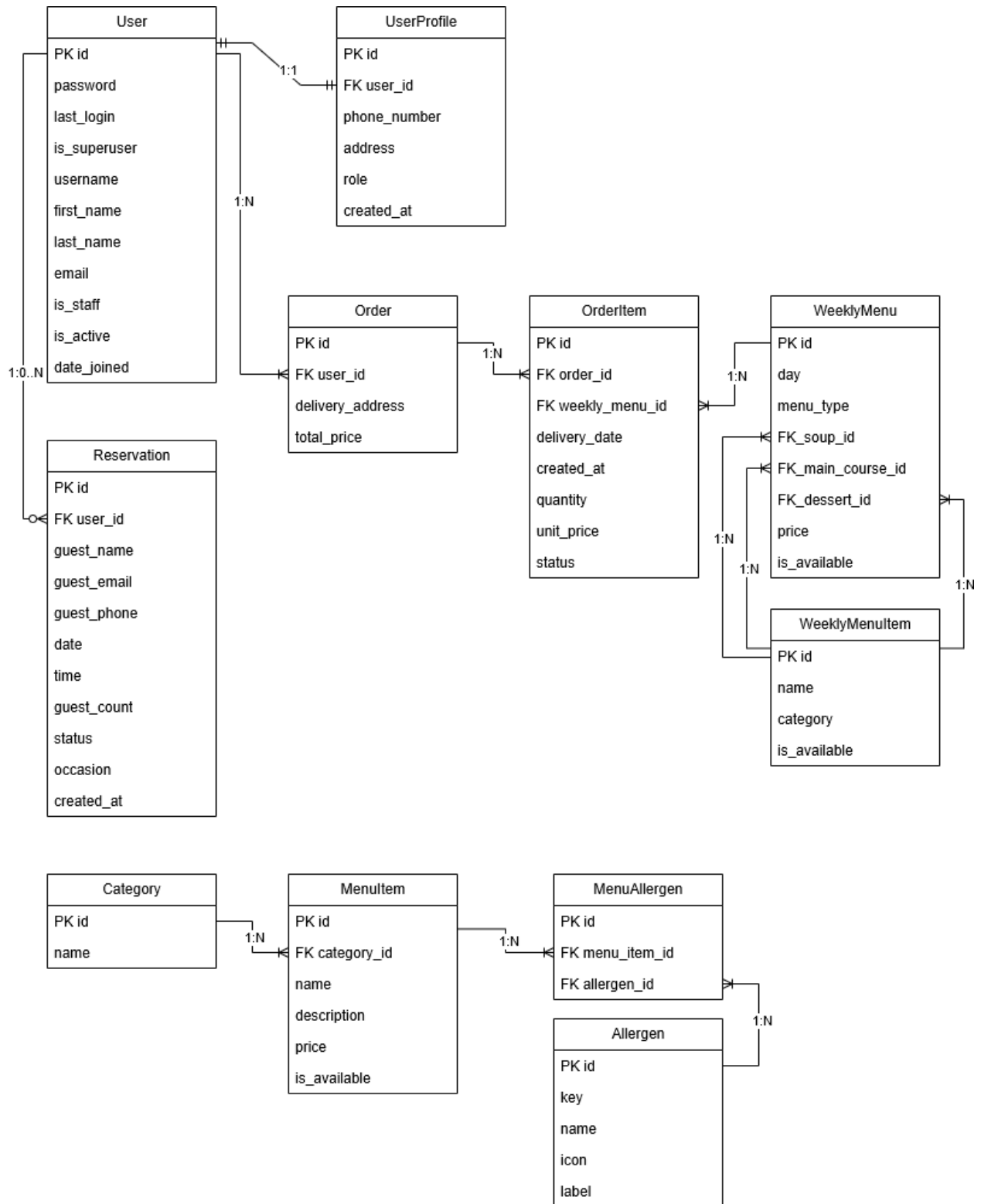
4. Adatmodell és adatbázis-tervezés

Az RMS fejlesztése során relációs adatbázist alkalmaztunk, amelyhez a Django ORM-en keresztül férünk hozzá. A fejlesztési környezetben SQLite adatbázismotort használtunk, amely a projekt méretéhez megfelelő megoldást jelentett, és a Django natívan támogatja. Az adatmodell kialakítását a rendszer működésének átgondolásával kezdtük. Első lépésként meghatároztuk, hogy a webalkalmazás milyen adatokat kezel, azok hogyan kapcsolódnak egymáshoz, illetve mely információkat szükséges önálló entitásként tárolni. Törekedtünk arra, hogy minden logikailag elkülönülő adatkör saját adattáblában jelenjen meg. Ennek megfelelően külön modell készült például a felhasználók, a rendelések, a rendelési tételek, a heti menü, az étlap, az asztalfoglalások és az allergének kezelésére. Az egyes entitások közötti kapcsolatokat elsődleges és idegen kulcsok biztosítják. Az adatmodell kialakításának célja egy jól áttekinthető, könnyen bővíthető struktúra létrehozása volt, amely megfelelően támogatja a rendszer egyes funkcióinak adatkezelési igényeit.

4.1. ER diagram

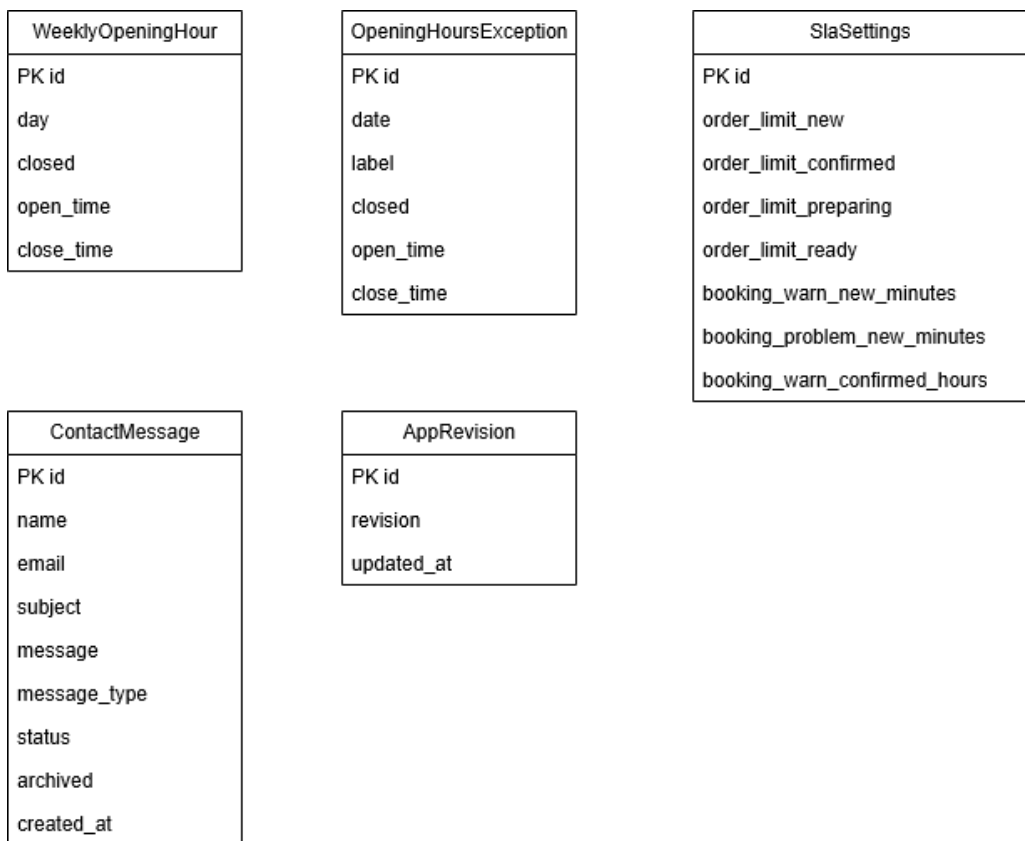
Az ER diagram az RMS adatmodelljének főbb entitásait, azok elsődleges és idegen kulcsait, valamint az entitások közötti kapcsolatokat szemlélteti. Az adatmodell áttekinthetősége érdekében a diagram két részben kerül bemutatásra.

4.1.1. Az RMS adatmodelljének kapcsolati része



Ábra: Az RMS adatmodelljének kapcsolati része

4.1.2. Az RMS adatmodelljének további adattáblái



Ábra: Az RMS adatmodelljének további adattáblái

4.2. Adattáblák áttekintése

Az alábbi táblázat az RMS adatmodelljében alkalmazott adattáblákat és azok elsődleges szerepét foglalja össze.

Adattábla	Szerepe
User	Felhasználói alap- és hitelesítési adatok
UserProfile	A felhasználóhoz kapcsolódó profiladatok
Reservation	Asztalfoglalások adatai
Order	Rendelések fejlécadatai
OrderItem	A rendelések tételei
WeeklyMenu	A heti menü napi A/B menüinek adatai
WeeklyMenuItem	A heti menü egyes fogásai
MenuItem	Állandó étlapon szereplő ételek
Category	Ételkategóriák
Allergen	Allergénadatok
MenuAllergen	Ételek és allergének kapcsolata
ContactMessage	Kapcsolatfelvételi üzenetek
WeeklyOpeningHour	Heti nyitvatartási adatok
OpeningHoursException	Egyedi nyitvatartási eltérések
SlaSettings	Dashboard időkorlátjainak beállításai
AppRevision	Technikai szinkronizációs verziószám

4.3. Adattáblák részletes bemutatása

Az adattáblák részletes bemutatása során az egyes modellek szerepét, fontosabb mezőit, valamint a kapcsolódó táblákkal fennálló kapcsolatokat ismertetjük. A bemutatás során elsősorban azokat az adatmodellben alkalmazott megoldásokat emeljük ki, amelyek a rendszer működése vagy későbbi bővíthetősége szempontjából meghatározóak.

4.3.1. Felhasználók és jogosultságok

User: A felhasználók alapvető adatait, valamint a Django beépített hitelesítési és jogosultságkezelési rendszeréhez kapcsolódó adatokat a Django **User** modelljében tároljuk. A modell elsődleges kulcsa az `id`, amelyhez a rendszer további, felhasználóhoz kapcsolódó adatai kapcsolódnak. A modell többek között a felhasználó e-mail címét, nevét, jelszavát, valamint a Django által biztosított jogosultsági és állapotinformációkat tartalmazza. A regisztráció során a felhasználó e-mail címe szolgál felhasználóazonosítónak, mivel a rendszerben a `username` mező értéke megegyezik az e-mail címmel.

UserProfile: A Django **User** modelljét saját **UserProfile** modellel egészítettük ki. Ennek oka, hogy a rendszer működéséhez a beépített felhasználói adatokon túl további információkra, például telefonszámra, címre és szerepkörre is szükség van. A két modell között egy-az-egyhez kapcsolatot hoztunk létre `OneToOneField` segítségével. A **UserProfile** elsődleges kulcsa mellett a `user_id` idegen kulcs hivatkozik a kapcsolódó felhasználóra. A szerepkört a `role` mező tárolja, amely a `customer`, `employee` és `admin` értékeket különbözteti meg. A projekt jelenlegi jogosultsági logikájában elsősorban a vendég és az adminisztrátori hozzáférés elkülönítése szükséges.

4.3.2. Rendelések

Order: Az **Order** modell egy rendeléshez kapcsolódó fejlécadatokat tartalmazza. Minden rendelés egy felhasználóhoz kapcsolódik a `user_id` idegen kulcson keresztül. A rendelésben szerepel többek között a szállítási cím, a teljes rendelési érték és a létrehozás időpontja.

OrderItem: A rendelés konkrét tartalmát az **OrderItem** modell tárolja. Egy rendeléshez több rendelési tétel tartozhat, ezért az **Order** és **OrderItem** között egy-a-többhöz kapcsolatot alakítottunk ki. A tétel a kapcsolódó heti menüt, a mennyiséget, a kiszállítás dátumát és a rendelés leadásakor érvényes egységárat is tárolja. Az egységárat nem kizárólag a **WeeklyMenu** aktuális árából számítjuk, hanem a rendelési tételben is eltároljuk. Ennek célja, hogy egy későbbi árváltozás ne módosítsa visszamenőlegesen a már leadott rendelések értékét. Az **OrderItem** saját státuszt is tartalmaz, így egy rendelés egyes tételei egymástól függetlenül is kezelhetők és feldolgozhatók.

4.3.3. Menü és étlap

MenuItem és Category: Az állandó étlap ételeit a **MenuItem** modell tárolja. Az ételek kategóriákba sorolását a külön **Category** modell biztosítja, amelyhez a **MenuItem** idegen kulccsal kapcsolódik. Így egy kategóriához több étel tartozhat, miközben egy étel csak egy kategóriához tartozik.

Allergen és MenuAllergen: Az allergének kezelésére külön **Allergen** modell készült. Egy ételhez több allergén tartozhat, és ugyanaz az allergén több különböző ételnél is megjelenhet. Emiatt a **MenuItem** és **Allergen** között több-a-többhöz kapcsolat áll fenn, amelyet a **MenuAllergen** köztes modell valósít meg. A köztes modell biztosítja, hogy az egyes étel–allergén párosok külön rekordként legyenek kezelhetők, és ugyanaz a páros ne kerülhessen többször az adatbázisba.

WeeklyMenu és WeeklyMenuItem: A heti menü adatmodelljét az állandó étlaptól elkülönítve alakítottuk ki. A kezdeti elképzelés szerint a heti menü az állandó étlap ételeiből állt volna össze, azonban a rendszer működésének és az éttermi folyamatnak a felülvizsgálata során ezt a megoldást nem tartottuk megfelelőnek. A heti menüben ugyanis az étlaptól eltérő, egyszerűbb fogások is szerepelhetnek, valamint a heti menü ára nem feltétlenül követi az egyes ételek állandó étlapon szereplő árait. Ennek megfelelően a heti menü fogásait a **WeeklyMenuItem** modellben külön kezeltük. A modell a fogás nevét, típusát – leves, főétel vagy desszert – és elérhetőségét tárolja. A **WeeklyMenu** modell egy adott nap A vagy B menüjét reprezentálja, és idegen kulcsokon keresztül három **WeeklyMenuItem** rekordhoz kapcsolódik: egy leveshez, egy főételhez és egy desszerthez. A `day` és `menu_type` mezők együttes egyedisége biztosítja, hogy egy adott naphoz ugyanazon menütípusból csak egy rekord tartozzon.

4.3.4. Foglalások és kapcsolatfelvétel

Reservation: Az **Reservation** modell az asztalfoglalások adatait tárolja. A foglalás opcionálisan kapcsolódhat regisztrált felhasználóhoz, ezért a `user_id` mező nullable. Ez lehetővé teszi, hogy a rendszer bejelentkezés nélkül is fogadjon foglalásokat. Ilyen esetben a vendég nevét, e-mail-címét és telefonszámát közvetlenül a foglalási rekord tartalmazza.

ContactMessage: A **ContactMessage** modell a publikus felületen keresztül érkező kapcsolatfelvételi üzenetek tárolására szolgál. A feladó alapvető adatai mellett az üzenet tárgyát, típusát és tartalmát, valamint feldolgozási állapotát is tárolja. Az `archived` mező lehetőséget biztosít arra, hogy az üzenet az aktív listából archiválható legyen anélkül, hogy fizikailag törölnénk az adatbázisból.

4.3.5. Nyitvatartás és rendszerbeállítások

WeeklyOpeningHour: A **WeeklyOpeningHour** modell az étterem általános, a hét napjaira vonatkozó nyitvatartási rendjét tárolja.

OpeningHoursException: Az ettől eltérő, konkrét dátumokra vonatkozó nyitvatartási rendet az **OpeningHoursException** modell kezeli. Ez lehetővé teszi például egy ünnepnap vagy egy rendkívüli zárvatartás rögzítését anélkül, hogy az általános heti nyitvatartást módosítani kellene.

SlaSettings: A **SlaSettings** modell a Dashboard működéséhez kapcsolódó időkorlátokat tárolja. Egyes rendelési státuszokhoz és foglalási állapotokhoz külön figyelmeztetési és problémaküszöbök tartoznak.

AppRevision: Az **AppRevision** modell technikai célokat szolgál. Nem üzleti adatot tárol, hanem egy globális verziószámot, amely jelzi a kliens számára, hogy az alkalmazás adatállapotában változás történt-e. A böngésző ennek változását figyelve szükség esetén újratölti a megjelenített adatokat.

4.4. Bővíthetőség

Az adatmodell kialakításánál törekedtünk arra, hogy az egyes funkcionális területek elkülönülten, önálló modellekben jelenjenek meg. Ennek köszönhetően egy új funkció bevezetése esetén nem szükséges a teljes adatstruktúrát átalakítani; az új adattípusok önálló modellekkel, illetve a meglévő modellekhez kialakított kapcsolatokkal illeszthetők a rendszerhez.

A jelenlegi struktúra több irányban is továbbfejleszthető. Az étlap például újabb, az ételekhez kapcsolódó információkkal egészíthető ki, míg az allergénekhez hasonlóan további, az ételek

jellemzőit leíró adatok is külön modellekben kezelhetők. A heti menü modellje további menütípusok vagy összetettebb menükonstrukciók támogatására is alkalmas lehet.

A rendelési struktúra szintén lehetőséget biztosít további funkciók bevezetésére. A jelenlegi **Order** és **OrderItem** elkülönítés mellett például fizetési adatok, szállítási információk vagy további rendelési státuszok is hozzáadhatók anélkül, hogy a rendelési tételek jelenlegi struktúráját alapvetően meg kellene változtatni.

A nyitvatartás kezelésének külön modelljei lehetővé teszik további kivételes időszakok kezelését is, míg a felhasználói adatok **User** és **UserProfile** modellekre történő szétválasztása további profiladatok hozzáadását teszi lehetővé a Django beépített felhasználói modelljének módosítása nélkül.

A felhasználói jogosultságok szintén tovább bővíthetők. A **UserProfile** modell `role` mezője jelenleg `customer`, `employee` és `admin` szerepköröket különböztet meg, így az adatmodell már lehetőséget biztosít egy külön alkalmazotti jogosultsági szint kezelésére. Ennek tényleges bevezetése esetén a szerepkörhöz tartozó hozzáférési szabályokat és a felhasználói felületeket is ennek megfelelően kell kialakítani.

5. Backend megvalósítása

5.1. A backend felépítése és az alkalmazások szerepe

Az RMS szerveroldali komponensét Django és Django REST Framework segítségével valósítottuk meg. A backend feladata a rendszer adatainak kezelése, az üzleti logika megvalósítása, valamint a frontend számára szükséges adatok és műveletek API-n keresztül történő biztosítása.

A Django projektet funkcionálisan elkülönített alkalmazásokra, úgynevezett appokra bontottuk. Az egyes appok egy-egy jól meghatározható feladatkört kezelnek, ezáltal a backend felépítése átláthatóbbá vált, és az egyes funkcionális egységek egymástól elkülönítve fejleszthetők.

Az RMS-ben az alábbi Django appokat hoztuk létre:

Django app	Feladata
users	Felhasználói adatok és a felhasználói profil kezelése
menu	Étlap, kategóriák, allergének és heti menük kezelése
orders	Rendelések és rendelési tételek kezelése
reservations	Asztalfoglalások kezelése
contact	Kapcsolatfelvételi üzenetek kezelése
opening_hours	Heti nyitvatartás és egyedi nyitvatartási eltérések kezelése
guest_portal	A bejelentkezett vendég saját rendeléseinek lekérdezése, módosítása és lemondása
sync	Az adatváltozások kliensoldali követését támogató verziószám kezelése

Az alkalmazások között a rendszer működéséhez szükséges kapcsolatokat alakítottunk ki. Az orders app például a menu appban definiált heti menü modelljeire támaszkodik, míg a guest_portal az orders app rendelési modelljeit használja fel a bejelentkezett vendég rendeléseinek megjelenítéséhez és módosításához. A feladatok ilyen elkülönítése lehetővé tette, hogy az egyes funkciókhoz kapcsolódó adatkezelés és szerveroldali logika jól meghatározható helyen jelenjen meg.

A backend működését több réteg együttműködése biztosítja. A Django modellek határozzák meg az adatbázissal kapcsolatos adatstruktúrát, a serializer réteg végzi az adatok JSON-formátumra alakítását és a kliens felől érkező adatok validációját, míg a view-k fogadják és dolgozzák fel az API-kéréseket. A következő alfejezetekben ezeket a megvalósítási elemeket részletesebben is bemutatjuk.

5.2. Az adatmodellek megvalósítása

A korábban bemutatott adatmodellt a Django ORM modelljeivel valósítottuk meg. Az egyes Django appok `models.py` fájljaiban definiáltuk az adatbázisban tárolt entitásokat, azok mezőit és kapcsolatait. A Django modellmezői lehetővé tették az adattípusok és alapvető korlátozások meghatározását, míg a kapcsolati mezőkkel az entitások közötti összefüggéseket képeztük le.

A modellek kialakításánál az adatmodellben meghatározott kapcsolatokhoz igazodtunk. A különböző kapcsolatokat a Django által biztosított `OneToOneField`, `ForeignKey` és `ManyToManyField` mezőtípusokkal valósítottuk meg.

A User és UserProfile közötti kapcsolat megvalósítása

A **User** és **UserProfile** modellek közötti egy-az-egyhez kapcsolatot a **UserProfile** modellben definiált `OneToOneField` valósítja meg:

```
user = models.OneToOneField(
    User,
    on_delete=models.CASCADE,
    related_name='profile'
)
```

A mező a Django beépített **User** modelljére hivatkozik. Az `on_delete=models.CASCADE` beállítás azt eredményezi, hogy a felhasználó törlésekor a hozzá tartozó profil is törlődik. A `related_name='profile'` segítségével a felhasználó objektumán keresztül közvetlenül elérhetővé válik a hozzá tartozó profil.

Az Order és OrderItem kapcsolatának megvalósítása

Az **Order** és **OrderItem** modellek között egy-a-többhöz kapcsolatot alakítottunk ki, mivel egy rendeléshez több rendelési tétel is tartozhat. A kapcsolatot az **OrderItem** modellben definiált `ForeignKey` valósítja meg:

```
order = models.ForeignKey(
    Order,
    on_delete=models.CASCADE,
    related_name='items'
)
```

A `ForeignKey` az **OrderItem** oldalán található, és a kapcsolódó **Order** rekordra hivatkozik. Az `on_delete=models.CASCADE` ebben az esetben azt biztosítja, hogy egy rendelés törlésekor a hozzá tartozó rendelési tételek is törlődjenek. A `related_name='items'` segítségével a rendelésből közvetlenül elérhetők a hozzá tartozó tételek.

Az **OrderItem** modell a rendelési tételhez tartozó heti menüt is `ForeignKey` kapcsolaton keresztül tárolja:

```
weekly_menu = models.ForeignKey(
    WeeklyMenu,
    on_delete=models.PROTECT
)
```

Ennek megfelelően minden rendelési tétel egy konkrét **WeeklyMenu** rekordhoz kapcsolódik. A `PROTECT` törlési szabály biztosítja, hogy olyan heti menü ne legyen törölhető, amelyre már rendelési tétel hivatkozik. Így a korábbi rendelésekhez tartozó menüadatok kapcsolata nem szüntethető meg a kapcsolódó heti menü véletlen törlésével.

Az **OrderItem** modellben az egységárat is külön tároljuk:

```
unit_price = models.DecimalField(
    max_digits=8,
    decimal_places=2
)
```

Az érték a rendelés létrehozásakor kerül meghatározásra a kiválasztott heti menü aktuális ára alapján. Ennek köszönhetően egy későbbi árváltozás nem módosítja a korábban leadott rendelésben szereplő összeget.

A MenuItem és Allergen közötti több-a-többhöz kapcsolat megvalósítása

Az étlap ételei és az allergének között több-a-többhöz kapcsolatot alakítottunk ki. Egy ételhez több allergén tartozhat, ugyanakkor egy allergén több ételnél is megjelenhet. A kapcsolatot `ManyToManyField` segítségével valósítottuk meg, amelyhez saját köztes modellt, a **MenuAllergen** modellt használtuk:

```
allergens = models.ManyToManyField(
    Allergen,
    through='MenuAllergen',
    blank=True
)
```

```
menu_item = models.ForeignKey(
    MenuItem,
    on_delete=models.CASCADE
)
```

```
allergen = models.ForeignKey(
    Allergen,
    on_delete=models.CASCADE
)
```

A `through='MenuAllergen'` beállítással megadtuk, hogy a több-a-többhöz kapcsolatot a saját **MenuAllergen** modell valósítsa meg. A köztes modell két `ForeignKey` mezőn keresztül kapcsolja össze az ételt és az allergént. Ez a megoldás lehetővé tette a kapcsolat önálló kezelését, valamint azt is, hogy egy adott étel és allergén párosa csak egyszer szerepelhessen az adatbázisban.

```
class Meta:
    unique_together = ('menu_item', 'allergen')
```

A `unique_together` beállítás megakadályozza, hogy ugyanahhoz az ételhez ugyanaz az allergén többször is hozzárendelhető legyen.

A **WeeklyMenu** fogásainak kapcsolatai és egyediségük biztosítása

A **WeeklyMenu** modell kialakítása során egy napi A vagy B menü három különálló fogásból állhat: levesből, főételből és desszertből. Mindhárom fogást külön `ForeignKey` kapcsolja a **WeeklyMenuItem** modellhez.

```
soup = models.ForeignKey(
    WeeklyMenuItem,
    on_delete=models.PROTECT,
    related_name='weekly_soups'
)

main_course = models.ForeignKey(
    WeeklyMenuItem,
    on_delete=models.PROTECT,
    related_name='weekly_mains'
)

dessert = models.ForeignKey(
    WeeklyMenuItem,
    on_delete=models.PROTECT,
    related_name='weekly_desserts'
)
```

A három külön kapcsolat lehetővé teszi, hogy a heti menü egyértelműen meghatározza, melyik fogás kerül levesként, főételként vagy desszertként a menübe. A `related_name` értékekkel a kapcsolódó fogások a megfelelő szerepkör alapján is elérhetők.

A `PROTECT` törlési szabály megakadályozza, hogy olyan **WeeklyMenuItem** rekord kerüljön törlésre, amelyre már létező heti menü hivatkozik.

```
class Meta:
    unique_together = ('day', 'menu_type')
```

A modellben egy további korlátozást is alkalmaztunk: egy adott naphoz és menütípushoz csak egy **WeeklyMenu** rekord tartozhat. Ezt a `day` és `menu_type` mezők együttes egyediségének biztosításával valósítottuk meg.

5.3. API réteg és végpontok

Az RMS backend és a frontend közötti adatcseréjét Django REST Framework segítségével valósítottuk meg. Az API-rétegben a serializer és a view-k együttműködése biztosítja a kliens felől érkező adatok feldolgozását, azok ellenőrzését, valamint az adatbázisból lekért adatok frontend számára megfelelő formában történő visszaadását. A gyakori műveletekhez a Django REST Framework által biztosított generikus nézeteket alkalmaztuk, amelyekre építve a rendszer saját működéséhez kapcsolódó lekérdezéseket és validációkat valósítottuk meg.

Serializer és generikus view-k

A serializer réteg feladata az adatbázisban található modellek adatainak JSON-formátumra alakítása, valamint a kliens által küldött adatok ellenőrzése. Az adatok lekérdezéséhez és megjelenítéséhez, illetve létrehozásához külön serializer-megoldásokat is alkalmaztunk, így a frontend számára mindig az adott művelethez szükséges adatok kerülnek továbbításra.

Erre példaként az **OrderItemSerializer** említhető, amely a rendelési tételek mellett a kapcsolódó heti menü adataiból is előállít a frontend számára szükséges információkat.

```
class OrderItemSerializer(serializers.ModelSerializer):
    menu_type = serializers.CharField(
        source="weekly_menu.menu_type",
        read_only=True
    )
    day = serializers.SerializerMethodField()
    menu = serializers.SerializerMethodField()
    menu_details = serializers.SerializerMethodField()
    class Meta:
        model = OrderItem
        fields = [
            "id",
            "weekly_menu",
            "delivery_date",
            "created_at",
            "quantity",
            "unit_price",
            "menu_type",
            "day",
            "menu",
            "menu_details",
            "status",
        ]
```

A serializer így nem kizárólag a modell közvetlenül tárolt mezőit adja vissza. A `menu_type` például a kapcsolódó **WeeklyMenu** modellből származik, míg a `day`, `menu` és `menu_details` mezőket a serializer állítja elő. Ezzel a frontend számára olyan, összesített adatszerkezetet biztosítottunk, amely a rendelés megjelenítéséhez szükséges adatokat egyetlen válaszban tartalmazza.

A gyakori API-műveletek megvalósításához a Django REST Framework generikus nézeteit használtuk. Ezek az alapvető műveletekhez szükséges működést biztosítják, így a saját kódban elsősorban a rendszerhez kapcsolódó lekérdezéseket és egyedi logikát kellett megadnunk. Például az **OrderListAPIView** a `ListAPIView` osztályra épül:

```
class OrderListAPIView(generics.ListAPIView):
    serializer_class = OrderSerializer
    def get_queryset(self):
        return (
            Order.objects
            .select_related("user", "user__profile")
            .prefetch_related("items__weekly_menu")
            .order_by("-id")
        )
```

A `get_queryset()` metódusban meghatároztuk, hogy a dashboard számára mely rendelések kerüljenek lekérdezésre. A `select_related()` és `prefetch_related()` használatával a rendeléshez kapcsolódó felhasználói, profil- és heti menüadatokat is előre betöltjük. Ez csökkenti a kapcsolódó adatok feldolgozásához szükséges adatbázis-lekérdezések számát.

Egy rendelés létrehozásának megvalósítása

Az új rendelés létrehozására a `POST /api/orders/create/` végpontot használjuk. A kérés feldolgozása során az `OrderCreateSerializer` először ellenőrzi a beérkező adatokat, majd az érvényes adatok alapján létrehozza a rendelést és annak tételeit.

A rendelési tétel létrehozásakor a backend a kiválasztott **WeeklyMenu** aktuális árából határozza meg a `unit_price` értékét:

```
weekly_menu = WeeklyMenu.objects.get(
    id=item_data["weekly_menu"]
)
quantity = item_data["quantity"]
unit_price = weekly_menu.price
OrderItem.objects.create(
    order=order,
    weekly_menu=weekly_menu,
    delivery_date=item_data["delivery_date"],
    quantity=quantity,
    unit_price=unit_price,
)
```

A `WeeklyMenu` rekord azonosítója alapján a backend lekéri a kiválasztott menüt, majd annak aktuális árából beállítja a rendelési tétel egységárát. Az ár így külön is rögzítésre kerül az **OrderItem** modellben, ezért egy későbbi árváltozás nem módosítja a korábban leadott rendelésben szereplő árat.

A rendelés létrehozása során az egyes tételek mennyiségének és egységárának felhasználásával a backend a rendelés teljes értékét is meghatározza. A létrehozott **Order** és a hozzá tartozó

OrderItem rekordok ezt követően a serializeren keresztül kerülnek visszaadásra a frontend számára.

5.4. Autentikáció

A rendszernek tudnia kell, hogy egy API-kérés bejelentkezett admintól, vendégtől, vagy senkitől jön. Erre munkamenet-alapú bejelentkezést használunk (Django session). A belépéstől a lekérésig és a módosításig négy lépés következik — ezt foglalja össze az alábbi ábra.



Ábra: Session autentikáció működése (négy lépés)

A bejelentkezés (`POST /api/auth/login/`) után a szerver munkamenetet (session) hoz létre, és a böngésző kap egy sessionid nevű cookie-t.

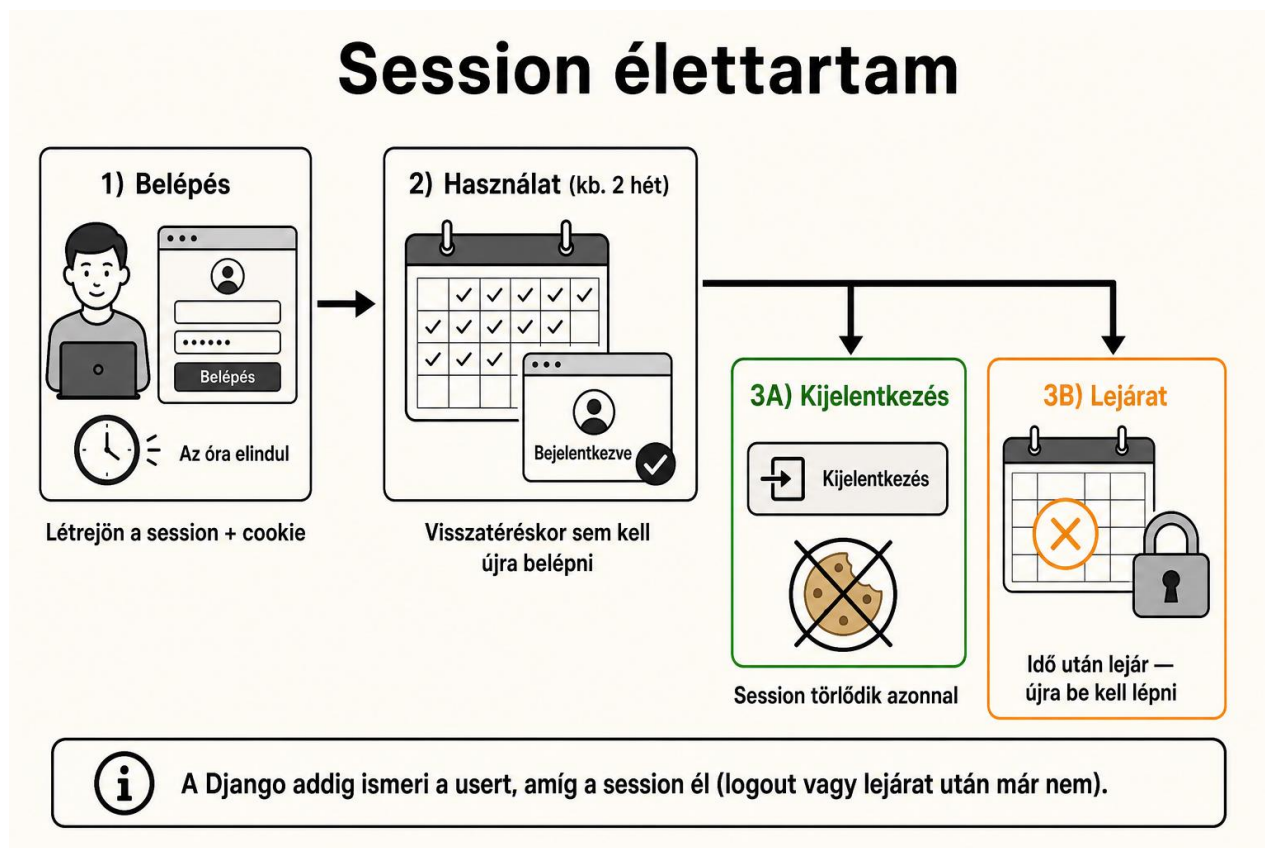
A későbbi kéréseknél a böngésző ezt a cookie-t automatikusan elküldi. Ebből ismeri fel a Django, ki van bejelentkezve. A Django REST Framework (DRF — a REST API-t kezelő Django-kiegészítő) ebből tölti ki a `request.user` mezőt, tehát a kód tudja, melyik felhasználóról van szó.

A session élettartamáról a következők fontosak:

- Session élettartam: megadja, meddig marad érvényes a munkamenet.
- Alapértelmezés: két hét (a Django saját alapbeállítása).
- Állítás: a `settings.py`-ban a `SESSION_COOKIE_AGE` adja meg az élettartamot másodpercben; például $86400 = 1$ nap (az alap két hét helyett).
- Miért számít: rövidebb élettartamnál hamarabb kell újra belépni — szigorúbb a biztonság, de kevesebb a kényelem.

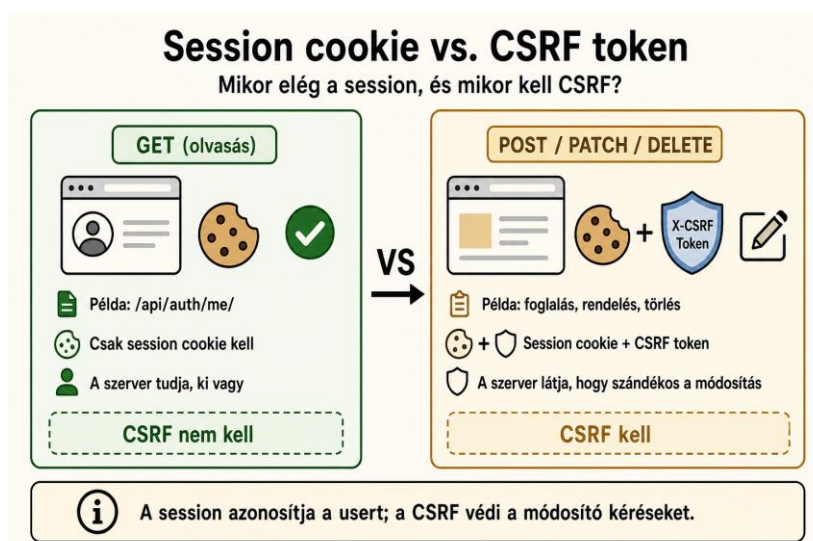
- Vége: kijelentkezéskor (POST /api/auth/logout/) a session azonnal törlődik; lejárat után újra be kell jelentkezni. Amíg a cookie és a szerver oldali session él, visszatéréskor a felhasználó bejelentkezve marad.

Ezt az időbeli működést mutatja a következő ábra.



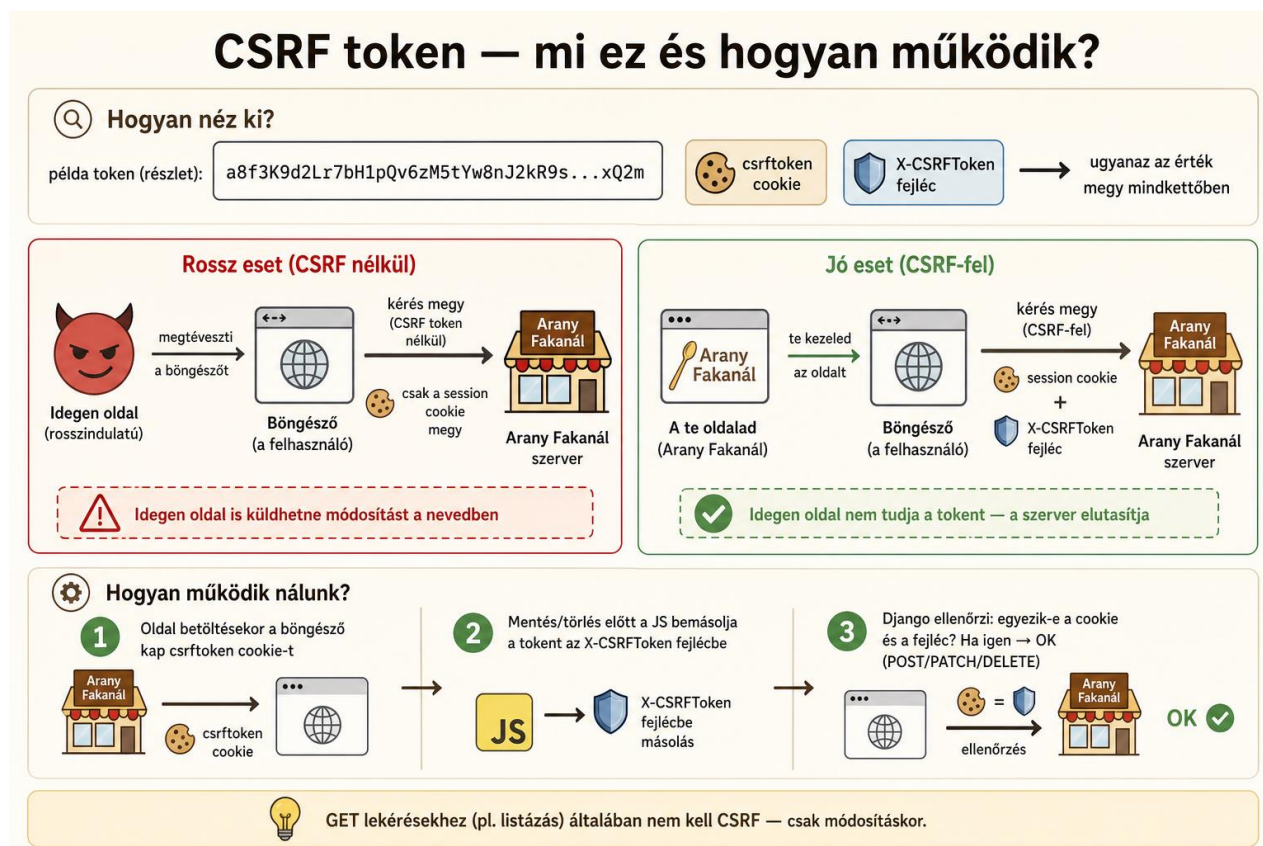
Ábra: Session élettartam (használat, kijelentkezés vagy lejárat)

A session azonosítja a felhasználót, de nem minden kéréshez ugyanaz kell. Olvasáskor (GET, például /api/auth/me/) általában elég a session cookie; ha adatot módosítunk (POST, PATCH, DELETE), CSRF token is szükséges. Ezt a különbséget mutatja a következő ábra.



Ábra: Mikor elég a session cookie (GET), és mikor kell CSRF token is (POST / PATCH / DELETE)

A session cookie-t a böngésző automatikusan küldi, ezért egy idegen oldal is megpróbálhatja kérését indítani a belépett felhasználó nevében. A CSRF token ezt akadályozza meg: a saját oldal a csrf token cookie értékét az X-CSRFToken fejlécbe teszi, a Django pedig csak akkor fogadja el a módosító kérést, ha ez egyezik. Ezt mutatja a következő ábra.



Ábra: CSRF token — megjelenés és működés

A session azért illik ide, mert a felület böngészőben fut, és a cookie-val együtt kezelhető a CSRF is. A regisztráció, a login, a logout és a GET /api/auth/me/ a users appban van. Az autentikáció csak az azonosságot állapítja meg (ki vagy); a szerepkör szerinti hozzáférést a jogosultságkezelés dönti el.

5.5. Jogosultságkezelés

Az autentikáció azt mondja meg: ki vagy. A jogosultságkezelés azt dönti el: mit szabad tenned. Ha valaki be van jelentkezve, a szerver tudja, melyik felhasználóról van szó (ezt a kódban request.user), és a szerepköre alapján engedi vagy tiltja a műveleteket.

A szerepkört a UserProfile.role mező tárolja. Három fontos érték van:

- customer — regisztrált vendég: használhatja a nyilvános oldalakat és a vendégközpontot.
- admin — éttermi admin: elérheti a dashboardot és az admin API-kat.
- employee — alkalmazott: a modellben már létezik, de külön felület jelenleg nincs hozzá (ez továbbfejlesztési terület).

A jogosultságot két lépésben érdemes nézni. Először: egyáltalán eléri-e a kérés azt az API-címet (végpont szint). Másodszor: ha eléri, milyen adatot kap vissza — mindent, vagy csak a sajátját (adat szint).

1. Végpont szint — bejöhet-e a kérés?

Itt a Django REST Framework (DRF) szabályai döntenek el címről címre, ki hívhatja az adott API-t. Néhány példa:

- Login és regisztráció — bárki hívhatja, bejelentkezés nélkül is (AllowAny).
- Logout — csak az hívhatja, aki már be van jelentkezve (IsAuthenticated).
- Vendégközpont API — bejelentkezett vendég hívhatja; az admin is beléphet ide (IsGuestPortalUser).
- Dashboard és admin listák — csak admin hívhatja (is_app_admin / role == admin). Bejelentkezés nélkül ezek a címek nem elérhetők.

2. Adat szint — mit kap vissza, ha már bejöhet?

Az, hogy valaki be van jelentkezve, még nem jelenti, hogy mindent láthat. Ugyanarra a témára (például rendelések) más lista jár az adminnak és más a vendégnek:

- Admin — a GET /api/orders/ az összes rendelést visszaadja.
- Vendég — a /api/guest-portal/orders/ és a /api/guest-portal/reservations/ csak a saját rendeléseit és foglalásait adja vissza. A guest_portal a bejelentkezett felhasználóra szűr, ezért más vendég adatait nem kapja meg.

Bejelentkezés nélkül ilyen „saját lista” API-t nem is lehet lekérni: nincs bejelentkezett felhasználó, amihez a saját rendeléseket vagy foglalásokat hozzá lehetne kötni. Aki nincs belépve, a nyilvános oldalakat böngészheti (étlap, foglaló űrlap), de admin adatokat és mások ügyeit nem kapja meg a szervertől.

Emellett státuszfüggő szabályok is vannak. A válaszban megjelenhet például a can_edit és a can_cancel (igaz vagy hamis). Ezekből tudja a frontend, melyik gombot mutassa. A vendég csak „Új” státuszú tételt vagy még függőben lévő (pending) foglalást tud módosítani vagy lemondani. Ami már továbbment (például visszaigazolt foglalás), azt a szerver nem engedi — akkor sem, ha a felhasználó be van jelentkezve.

6. Frontend megvalósítása

Ebben a fejezetben azt szeretnénk bemutatni, hogy hogyan épül fel ténylegesen a frontend a projektben. Sorra vesszük a fájlok mappáinak szerkezetét, az egyes oldalak felépítését, a rendszer közös felületi elemeit, a JavaScript kód modulokra bontását, valamint a reszponzív megjelenítés és a felhasználói élmény kialakítása mögötti megoldásokat. A backend és a frontend közötti kapcsolatot, illetve a hibrid megjelenítési modellt (szerver oldali sablon + JavaScript által betöltött adatok) a 3.4. alfejezet már érintette - ez a fejezet ehhez képest a konkrét megvalósításra fókuszál.

6.1. Mappastruktúra

A frontend két, egymástól jól elkülönített helyen található a projektben, mivel ez a két rész eltérő szerepet tölt be: a Django szerver oldalon állítja össze és rendereli a HTML-vázatot, miközben a statikus állományok - CSS, JavaScript, képek, ikonok - változatlan formában, közvetlenül a böngészőhöz kerülnek. Ennek megfelelően a sablonok a backend/templates/, a statikus fájlok pedig a frontend/static/ mappában helyezkednek el. A kettő közötti kapcsolatot a settings.py

STATICFILES_DIRS beállítása teremti meg, amely a {% static %} sablon-taggel és a /static/... URL-en keresztül teszi elérhetővé a statikus tartalmat.

A mappaszerkezet emellett a rendszer három fő funkcionális egységét (lásd 1.4.) tükrözi vissza: a publikus felület fájljai a gyökérben, a Dashboardé a dashboard/, a Vendégközponté pedig a guest-portal/ almappákban találhatók - mind a sablonok, mind a statikus fájlok szintjén. Az alábbi ábra ezt a felosztást mutatja:

```
backend/templates/
├── base.html                # közös váz a publikus oldalakhoz
├── homepage.html           # főoldal
├── etlap.html              # állandó étlap
├── asztalfoglalas.html     # asztalfoglalás
├── components/             # újrahasznosítható HTML-részletek
│   ├── navbar.html
│   ├── login.html
│   ├── cart.html
│   ├── profile-panel.html
│   ├── privacy-modal.html
│   └── privacy-consent.html
├── dashboard/
│   ├── index.html          # admin dashboard (önálló oldal)
│   └── partials/
├── guest-portal/
│   └── index.html          # vendégközpont (önálló oldal)
frontend/static/
├── css/                    # publikus oldalak + közös stílusok (components, toast,
privacy-modal)
├── js/                     # publikus oldalak + közös JS modulok (core/api.js, toast.js...)
├── dashboard/
│   ├── css/
│   └── js/
├── guest-portal/
│   ├── css/
│   └── js/
└── images/
```



```
└─ icons/          # allergén ikonok
```

Ezzel a következetes elnevezéssel igyekeztünk megkönnyíteni, hogy egy adott funkcióhoz tartozó fájlokat gyorsan meg lehessen találni.

6.2. Oldalak felépítése

Az oldalak felépítésénél két különböző mintát alkalmaztunk, attól függően, hogy publikus vagy belépéshez kötött felületről van szó.

6.2.1. Publikus oldalak - sablonok öröklése

A főoldal, az étlap és az asztalfoglalás oldalak a Django sablonöröklés (*template inheritance*) mechanizmusát használják. A `base.html` tartalmazza a közös vázat: a `<head>` részt, a navigációs sávot, a bejelentkezési és kosár komponenseket, illetve a közösen használt JavaScript fájlok betöltését. A konkrét oldalak (pl. `homepage.html`) például az `{% extends "base.html" %}` utasítással öröklik ezt a vázat, majd a `{% block %}` szakaszokkal (`title`, `css`, `content`, `scripts`) csak a saját, egyedi tartalmukat adják hozzá:

```
{% extends "base.html" %}
{% block title %}Arany Fakanál Étterem - Szilváskút{% endblock %}
{% block css %}
    <link rel="stylesheet" href="{% static 'css/homepage.css' %}" />
{% endblock %}
{% block content %}
    <header class="hero">...</header>
{% endblock %}
```

Ábra: Részlet a `homepage.html`-ből — sablonörökléssel csak az egyedi tartalom kerül az oldalba

Ennek köszönhetően a navbar, a bejelentkező ablak és a kosár HTML-je egyetlen helyen létezik. Így nem kell minden oldalon külön-külön megismételni és egy esetleges módosítás pedig automatikusan minden publikus oldalon érvényesülni tud.

6.2.2. Dashboard és Vendégközpont - önálló oldalak

A Dashboard és a Vendégközpont felépítése nagyrészt eltér a publikus oldalakétól. Az admin felület oldalsávot használ azért, hogy a személyzet munkáját megkönnyítse a navigáció közben. A Vendégközpont pedig saját navigációval rendelkezik. Emiatt egyik sem a `base.html`-ből építkezik, hanem önálló `<head>` szekcióval és külön CSS/JS köteggel rendelkezik, amelyet a saját dashboard/, illetve guest-portal/ mappájából tölt be.

Mindhárom felület megjelenítését a `config/views.py`-ban definiált nézetfüggvények szolgálják ki, amelyek a megfelelő sablont renderelik. A Dashboard és a Vendégközpont nézetei emellett jogosultságot is ellenőriznek (`is_app_admin`, illetve `can_access_guest_portal` - lásd 6.5.), és jogosulatlan hozzáférés esetén visszairányítják a felhasználót a főoldalra:

```
def dashboard(request):
    if not is_app_admin(request.user):
        return redirect("/")
```

```
return render(request, "dashboard/index.html")
```

Ábra: Részlet a dashboard nézetfüggvényből — jogosultság-ellenőrzés után a sablon renderelése

6.3. Közös felületi elemek

A publikus oldalakon visszatérő felületi elemeket külön HTML-részletekbe (*partial*) szerveztük ki, a templates/components/ mappában. A navbar, a bejelentkező ablak, a kosár, a profilmenü és az adatkezelési tájékoztató felugró ablaka mind a base.html-ből épül be {% include %}-dal — tehát minden publikus oldalon ott vannak, csak rejtve, amíg meg nem nyitja őket a felhasználó. Egy kivétel van: a privacy-consent.html, vagyis az elfogadó jelölőnégyzet. Ezt nem a base.html illeszti be, hanem közvetlenül azok az űrlapok, amelyek ténylegesen használják - a regisztráció, az asztalfoglalás és a kapcsolatfelvétel.

Komponens	Szerepe
navbar.html	Logó, főmenü, hamburger-gomb mobil nézetben, profil- és kosárikon
login.html	Bejelentkezés / regisztráció felugró ablak (modal)
cart.html	Kosár kihúzható panel (drawer)
profile-panel.html	Bejelentkezett felhasználó menüje: mobilon alulról kicsúszó lap, asztali nézetben lenyíló menü
privacy-modal.html	Adatkezelési tájékoztató felugró ablaka, minden publikus oldalon jelen van
privacy-consent.html	Elfogadó jelölőnégyzet, amit több űrlap (regisztráció, foglalás, kapcsolat) újrahasznosít

Táblázat: A publikus felület közös komponensei

A profil-gomb viselkedése bejelentkezéskor vált: a JS lecseréli a célját a bejelentkező ablakról a saját felületre mutató linkre - adminnál Dashboardra, vendégnél Vendégközpont. A kosár ikonján lévő számláló ugyanígy JS-ből frissül, ahányszor tétel kerül a kosárba.

A visszajelzésekhez egy közös toast-rendszert alakítottunk ki (toast.js), amit a window.showToast(üzenet, típus) globális függvényen keresztül minden felület használ - a publikus oldalak, a Dashboard és a Vendégközpont egyaránt. Sikeres mentésnél, hibánál vagy törlésnél

jelenik meg, típustól függő ikonnal és színnel; törlésnél opcionálisan egy „Visszavonás” gombbal is.

6.4. JavaScript modulok

A frontend JavaScript kódját külön keretrendszer nélkül írtuk meg, natív modulokra bontva. Funkciónként külön fájl tartozik egy-egy felületi részhez. A sablonok `<script defer>` címkékkel töltik be ezeket, méghozzá meghatározott sorrendben - ez azért fontos, mert pl. a `login.js` vagy a `cart.js` a korábban betöltött `core/api.js` függvényeire épül.

A rendszer alapját a `core/api.js` adja. Ez biztosítja minden fetch-hívásnak a session cookie küldését és a CSRF-védelmet (`apiRequest()`), illetve a bejelentkezési állapot lekérdezését (`checkAuthSession()`). A publikus oldalak, a Dashboard és a Vendégközpont kódja egyaránt ezt a modult használja - nem kellett a CSRF-kezelést többször megírni.

Terület	Modul	Feladata
Publikus oldalak	<code>navbar.js</code>	Görgetés-figyelés, mobil hamburger menü, aktív link kiemelése
	<code>login.js</code>	Bejelentkezés / regisztráció, profilmenü
	<code>cart.js</code>	Kosárlogika (<code>localStorage</code> , felhasználónként elkülönítve)
	<code>weekly-menu.js</code> / <code>etlap.js</code>	Heti menü, illetve állandó étlap betöltése az API-ból
	<code>contact-form.js</code> / <code>booking-form-prefill.js</code>	Kapcsolatfelvételi és foglalási űrlapok kezelése, bejelentkezett adatok előtöltése
	<code>opening-hours.js</code>	Nyitvatartás lekérése és megjelenítése
	<code>live-sync.js</code>	Élő adatfrissítés (lásd 7.6. és 8.2.7.)

	privacy-modal.js	Adatkezelési tájékoztató modal nyitása / elfogadása
Dashboard	index.js	Belépési jogosultság ellenőrzése (csak admin maradhat az oldalon)
	dashboard.js	Áttekintő nézet: mai statisztikák, teendők, agenda, SLA-túllépés jelzése
	orders.js, bookings.js	Rendelések és foglalások kezelése
	menu-manager.js, weekly-menu-combobox.js	Étlap és heti menü szerkesztése
	messages.js, settings.js, sla-rules.js	Üzenetek, nyitvatartás és SLA-beállítások
	api-errors.js, date-format.js	Hibaüzenetek és dátumformázás a dashboardon
Vendégköz-pont	index.js, account.js	Oldalváz, navigáció, saját adatok szerkesztése
	orders.js, bookings.js, guest-portal-sync.js	Saját rendelések, foglalások és ezek élő frissítése

Táblázat: A frontend JavaScript moduljainak áttekintése terület szerint

Ez a felosztás a backend app-szerkezetét (6.1.) követi: egy adott funkcióhoz - pl. foglalások - a backend és a frontend oldalán is jól beazonosítható, önálló fájlok tartoznak. Ez megkönnyíti a hibakeresést és a bővítést is.

6.5. Reszponzív kialakítás

A rendszer minden oldalán jelen van a `<meta name="viewport" content="width=device-width, initial-scale=1.0">` beállítás. Enélkül mobilon a böngésző egy kicsinyített asztali nézetet mutatna, nem a tényleges képernyőszélességhez lenne a tartalom igazítva. A projekt 18 CSS fájljából 16

tartalmaz @media lekérdezést. A reszponzivitás tehát nem pár kiemelt oldalra korlátozódik, hanem gyakorlatilag mindenhol jelen van, ahol az szükséges.

A publikus felület fő töréspontjai 992px (tablet) és 768px (mobil) szélességnél vannak. Ez alatt a navigációs sáv a teljes menüsor helyett hamburger-ikonra vált, amelyre kattintva a nav-links lista legördül (navbar.js). A Dashboard és a Vendégközpont oldalsávja asztali nézetben összecukható, mobil nézetben pedig a képernyő szélére húzható panelként (*drawer*) jelenik meg, hogy szűkebb helyen is elérhető maradjon a navigáció.

A mozgásra érzékeny felhasználókra is gondoltunk. A prefers-reduced-motion médialekérdezést a CSS animációk és a JS-es sima görgetés (navbar.js) egyaránt figyeli, tehát ha valaki rendszerszinten kikapcsolta a mozgáshatásokat, azonnali, animáció nélküli váltásra áll át a rendszer.

6.6. Felhasználói élmény (UX)

A felhasználói élmény kialakításánál az alábbi szempontokat tartottuk szem előtt.

6.6.1. Egységes megjelenés

A Brand Guide-ban (2.1.) meghatározott színpalettát (arany, fekete, fehér tónusok) és betűtípusokat (Playfair Display, Charm, Inter) mindhárom terület — publikus felület, Dashboard, Vendégközpont a saját CSS-változóiban (:root) rögzíti, ugyanazokkal az értékekkel. Nem egy közös fájlból öröklík, hanem három helyen van szinkronban tartva, az eredmény viszont vizuálisan egységes felület.

6.6.2. Azonnali visszajelzés

A közös toast-rendszer minden fontosabb művelet (mentés, hiba, törlés) után rövid, jól látható visszajelzést ad. Így a felhasználó mindig tudja, hogy tényleg megtörtént-e, amit csinált.

6.6.3. Élő frissítés

A live-sync.js miatt nem kell kézzel frissíteni az oldalt ahhoz, hogy valaki lássa mások, például az admin módosításait. Ez főleg a rendelések és foglalások állapotának követésénél számít.

6.6.4. Kényelmi automatizmusok

Bejelentkezett felhasználónál a foglalási és kapcsolatfelvételi űrlapok automatikusan előtöltik a nevet, e-mail címet és telefonszámot, így ezeket nem kell újra begépelni.

6.6.5. Akadálymentesség

A navigációs és profilmenü elemeit aria-label, aria-expanded és role="menu" attribútumokkal láttuk el, hogy képernyőolvasóval is használhatók legyenek.

6.6.6. Szerepkör szerinti irányítás

A rendszer a bejelentkezett felhasználó szerepköre alapján irányítja a megfelelő felületre (vendég -> Vendégközpont, admin -> Dashboard). Jogosulatlan hozzáférés esetén a Vendégközpont csendben, a Dashboard pedig egy figyelmeztető ablak után irányít vissza a főoldalra.

7. A rendszer működése

7.1. Az alkalmazás által biztosított fő működési folyamatok

A rendszerben a vendég rendelhet, asztalt foglalhat és üzenetet küldhet; az admin a dashboardon kezeli ezeket. A fő folyamatok ezek — az ábra csak az áttekintést mutatja.



Ábra: Az alkalmazás fő működési folyamatai — áttekintés

- Regisztráció / bejelentkezés — e nélkül nincs rendelés leadás és vendégközpont; az admin felület is belépéshez kötött.
- Heti menü rendelés — A vendég az A vagy B menüt a kosárba teszi, majd bejelentkezve leadja a rendelést a kiszállítási cím megadásával.
- Asztalfoglalás — A vendég megadja a dátumot, az időpontot, a létszámot és az alkalom típusát. Az elérhető időpontok a nyitvatartásból jönnek; belépés nélkül is foglalható, bejelentkezve pedig a név, e-mail és telefon előtöltődik.
- Kapcsolatüzenet — A vendég üzenetet küld a kapcsolati űrlapon, az admin pedig a dashboardon látja és kezeli.
- Admin dashboard — Itt követhetők a rendelések, foglalások, a heti menü, az üzenetek, a nyitvatartás és az SLA beállítások.
- Vendégközpont — A bejelentkezett vendég itt látja a saját rendeléseit, foglalásait és adatait; amíg a tétel Új vagy függőben van, módosíthatja vagy lemondhatja.
- Élő frissítés — Ha egy másik ablakban megváltozik valami, a lista magától frissül a revision / live-sync mechanizmuson keresztül. Ennek nincs külön képernyője: ugyanazokon a listákon működik, mint a rendelések vagy a dashboard áttekintő — ha a vendég lead egy rendelést, az admin felületen rövidesen megjelenik, anélkül hogy kézzel frissítenéd az oldalt.

Az alábbi képernyőképek a fő működési folyamatokat mutatják be.

Regisztráció / bejelentkezés

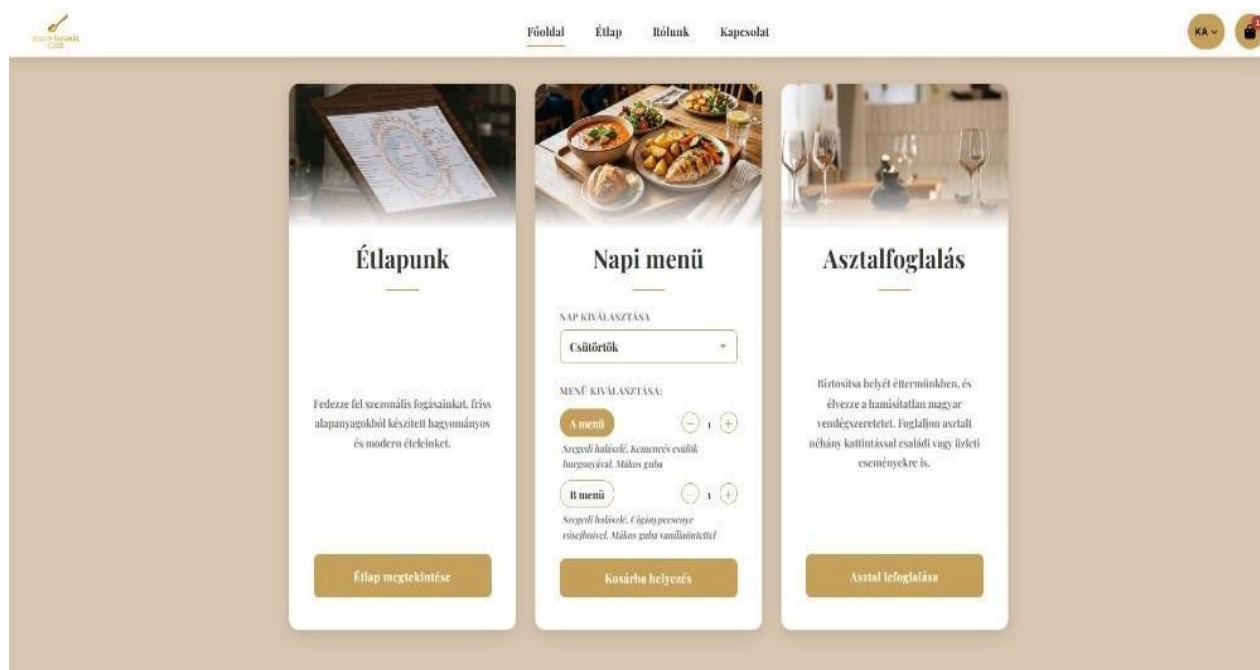
A nyilvános oldalakon a jobb felső profilikon nyitja a bejelentkező és a regisztrációs panelt. Új vendég előbb regisztrál, majd Kiss Anna e-mail címmel és jelszóval belép a fiókjába.



Ábra: Regisztráció / bejelentkezés — a jobb felső ikonnal indítható; új felhasználó előbb regisztrál, majd Kiss Anna belép a fiókjába

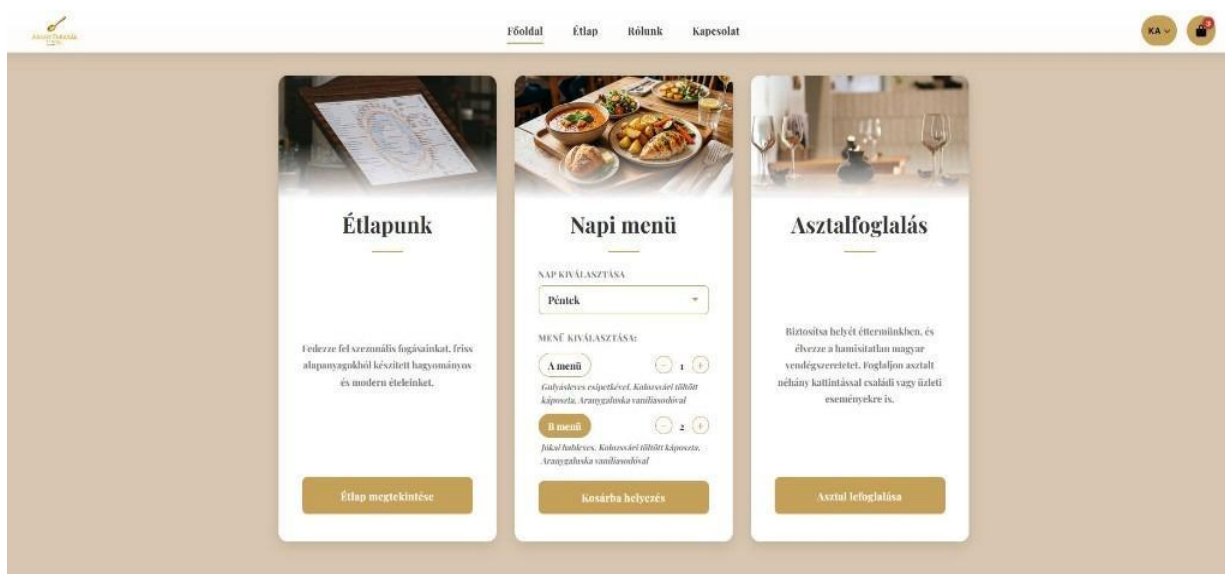
Heti menü rendelés

A főoldalon az aktuális hét A és B menüje választható napokra bontva. Kiss Anna csütörtökre az A menüt a kosárba teszi; a kosár ikonon azonnal látszik, hogy van benne tétel.



Ábra: Heti menü rendelés 1/3 — Kiss Anna (KA) az aktuális hét csütörtökére az A menüt rendeli; a kosár ikonon látszik, hogy már van benne tétel

Ugyanezen a felületen másik napra is lehet rendelni. Kiss Anna péntekre a B menüből két adagot választ, a kosárban így már két tétel van.



Ábra: Heti menü rendelés 2/3 — Kiss Anna péntekre a B menüből két adagot választ

A kosár tartalmát a vendég egy gombbal küldi el. A szerver létrehozza a rendelést, a dashboard pedig a live-sync miatt kb. két másodperc alatt megjeleníti — az adminnak nem kell frissítenie az oldalt.

Rendelés: vendég leadja → admin meglátja

VENDÉG – Kosár

Kosár

Csülkötök A menü 2026. aug. 6. 1990 Ft

Szegedi halászlé / Kemencés csülök burgonyával / Malacos guba

1

Péntek B menü 2026. aug. 7. 3090 Ft

Jókai bablevés / Rántott csirkemell káposzta / Aranygaluska vaníliaszóssal

2

Összesen 4500 Ft

Rendelés beküldése

ADMIN – Rendelések

0 Lezárt 2 Friss 0 Készlet 0 Kiszállítás 41 Lezárt

Keresés...

Név	Kapcsolat	Rendelés	Kiszállítás	Rend. idő	Státusz	Művelet
Kiss Anna	0620123654	Csülkötök A menü x3	2026. aug. 6.	6 perce	Új	
Kiss Anna	0620123654	Péntek B menü x2	2026. aug. 7.	6 perce	Új	
Kiss Anna	0620123654	Péntek A menü x3	2026. júl. 17.	21 napja	Készletben	
Kiss Anna	0620123654	Péntek B menü x3	2026. júl. 14.	23 napja	Készletben	
Kiss Anna	0620123654	Kedd A menü x1	2026. júl. 8.	28 napja	Készletben	
Kiss Anna	0620123654	Szombat A menü x2	2026. júl. 8.	28 napja	Készletben	
Kiss Anna	0620123654	Callimidi A menü x1	2026. júl. 9.	28 napja	Készletben	
Kiss Anna	0620123654	Péntek B menü x1	2026. júl. 0.	38 napja	Készletben	
Kiss Anna	0620123654	Hétfő A menü x1	2026. júl. 6.	30 napja	Készletben	
Kiss Anna	0620123654	Kedd B menü x1	2026. júl. 7.	26 napja	Készletben	

A vendég kiválasztja a menüt, majd elküldi a rendelést.

Az admin felületén ~2 másodpercen belül megjelenik az új rendelés (Új státuszban).

Ábra: Heti menü rendelés 3/3 — Kiss Anna elküldi a rendelést, amely kb. 2 másodperc alatt megjelenik az admin dashboardon (élő frissítés)

Asztalfoglalás

Az /asztalfoglalas/ oldalon a vendég megadja a dátumot, az időpontot, a létszámot és az alkalmat. Bejelentkezés után a név, e-mail és telefon előtöltődik, a foglalás pedig kb. két másodperc alatt megjelenik az admin listáján.

Asztalfoglalás

1. Vendég foglal

Foglalási adatok

Töltsd ki az alábbi mezőket és visszaigazoltat küldöm e-mailben.

TELJES NÉV: Kiss Anna
EMAIL CÍM: kissanna@gmail.com
TELEFONSZÁM: 0620123654
ALKALOM: Családi összejövetel
DÁTUM: 2026. 08. 05.
IDŐPONT: 18:00
SZEMÉLYEK SZÁMA: 2
MÉGJEGYZÉS: Asztal melletti asztalt kérünk, ha lehetséges.

2. Admin látja

Foglalások

1 2 4 12

Vendég	Kapcsolat	Résztvevők	Megj. időpont	Megjegyzés	Statusz	Műveletek
Kiss Anna	kissanna@gmail.com	Családi összejövetel	2026. aug. 5. 18:00	Asztal melletti asztalt kérünk, ha lehetséges.	Új	
Kiss Anna	kissanna@gmail.com	Születésnap	2026. aug. 4. 18:00		Visszaigazolt	
Kiss Anna	kissanna@gmail.com	Együtt	2026. júl. 28. 20:00	Kérem csendes asztalt, kérem, ha lehetséges.	Teljesítés	
Kiss Anna	kissanna@gmail.com	Családi összejövetel	2026. júl. 15. 18:00		Lemondás	
Kiss Anna	kissanna@gmail.com	Évforduló	2026. júl. 7. 18:00	zárk. asztal, garatol asztal.	Lemondás	
Kiss Anna	kissanna@gmail.com	Családi összejövetel	2026. júl. 4. 18:30	Születésnap, van megajándékozni.	Teljesítés	

~2 mp / élő frissítés

A vendég kitölti a foglalási adatokat, majd elküldi a foglalást.

Az admin felületén ~2 másodpercen belül megjelenik az új foglalás (Új státuszban).

Ábra: Asztalfoglalás — bejelentkezés után Kiss Anna adatai előtöltődnek; a foglalás kb. 2 másodperc alatt megjelenik az admin felületen (élő frissítés)

A foglalás a vendégközpontban is megjelenik, nem csak az admin felületén. Amíg Új státuszú, Kiss Anna a Foglalásaim listán szerkesztheti vagy lemondhatja.

Asztalfoglalás a vendégközpontban

1. VÉNDÉG – Foglalásaim

Foglalásaim

Asztalfoglalásaid kezelése egy helyen.

Módosítás és lemondás

A foglalást addig tudod szerkeszteni vagy lemondani, amíg státusza Új. Ha már visszaigazoltuk, kérjük, telefonon vagy e-mailben jelezd a változást.

Aktív (3) Lezárt (5)

2026. júl. 19. 20:00 • 5 fő	Visszaigazolt
2026. aug. 4. 18:00 • 2 fő	Új
2026. aug. 5. 18:00 • 2 fő	Új

Új státuszban

2. Módosítás / lemondás

Foglalásaim

Asztalfoglalásaid kezelése egy helyen.

Módosítás és lemondás

A foglalást addig tudod szerkeszteni vagy lemondani, amíg státusza Új. Ha már visszaigazoltuk, kérjük, telefonon vagy e-mailben jelezd a változást.

Aktív (3) Lezárt (5)

2026. júl. 19. 20:00 • 5 fő	Visszaigazolt
2026. aug. 4. 18:00 • 2 fő	Új
2026. aug. 5. 18:00 • 2 fő	Új

SZERDA
2026. aug. 5. 18:00 2 fő

ALKALOM
Családi összejövetel

TELEFON
0620123654

MÉGJEGYZÉS
Nincs megjegyzés

Létrehozva: 2026.08.05 16:48

Szerkesztés Lemondás

A foglalás a vendég saját Foglalásaim listáján is megjelenik.

Amíg a foglalás Új, Kiss Anna szerkesztheti vagy lemondhatja.

Ábra: Asztalfoglalás a vendégközpontban — a foglalás Kiss Anna Foglalásaim listáján is megjelenik; amíg Új státuszú, szerkeszthető vagy lemondható

Kapcsolatüzenet

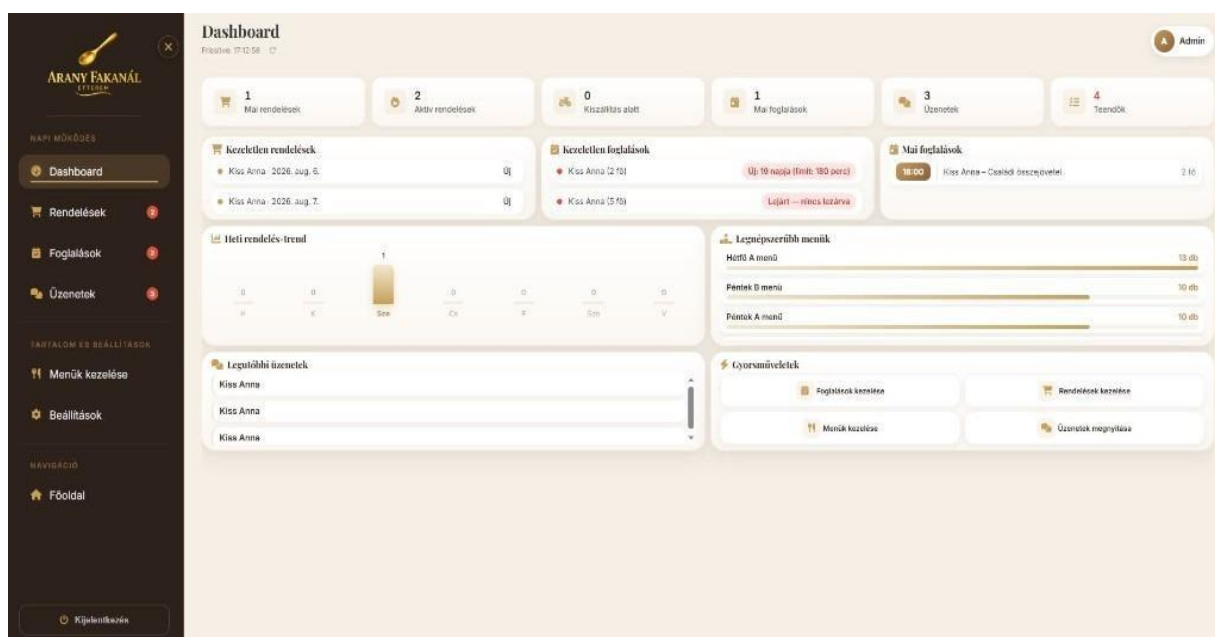
A főoldal kapcsolat szekciójában belépés nélkül is írható üzenet. Kiss Anna kitölti az űrlapot, elfogadja az adatkezelési tájékoztatót, és az üzenet a dashboardon jelenik meg.



Abra: Kapcsolatüzenet — Kiss Anna kitölti az űrlapot, elfogadja az adatkezelési tájékoztatót, majd az üzenet kb. 2 másodperc alatt megjelenik a dashboardon, ahol elolvasható (élő frissítés)

Admin dashboard

Az admin a /dashboard/ áttekintőjén látja a napi számokat és a teendőket. A kezeletlen rendelések között Kiss Anna frissen leadott tétele is szerepel, amint a live-sync átadta.



Abra: Admin dashboard 1/6 — áttekintő kezdőnézet; Kiss Anna rendelései a kezeletlen rendelések között jelennek meg

A rendelések szekcióban az admin látja a státuszokat, és egy legördülőből váltani is tud. Ha egy Új rendelés túl sokáig vár — például Kiss Annáé —, a sor pirossal jelzi.

Ábra: Admin dashboard 2/6 — rendelések kezelése; a státusz változtatható, és ha egy Új rendelés (pl. Kiss Annáé) túl sokáig vár, a sor pirossal jelzi

A foglalások külön szekcióban kezelhetők: dátum, létszám, alkalom és státusz egy táblázatban. Innen fogadható el vagy utasítható el a foglalás, és a változás a vendégközpontban is lekövethető.

Ábra: Admin dashboard 3/6 — foglalások kezelése

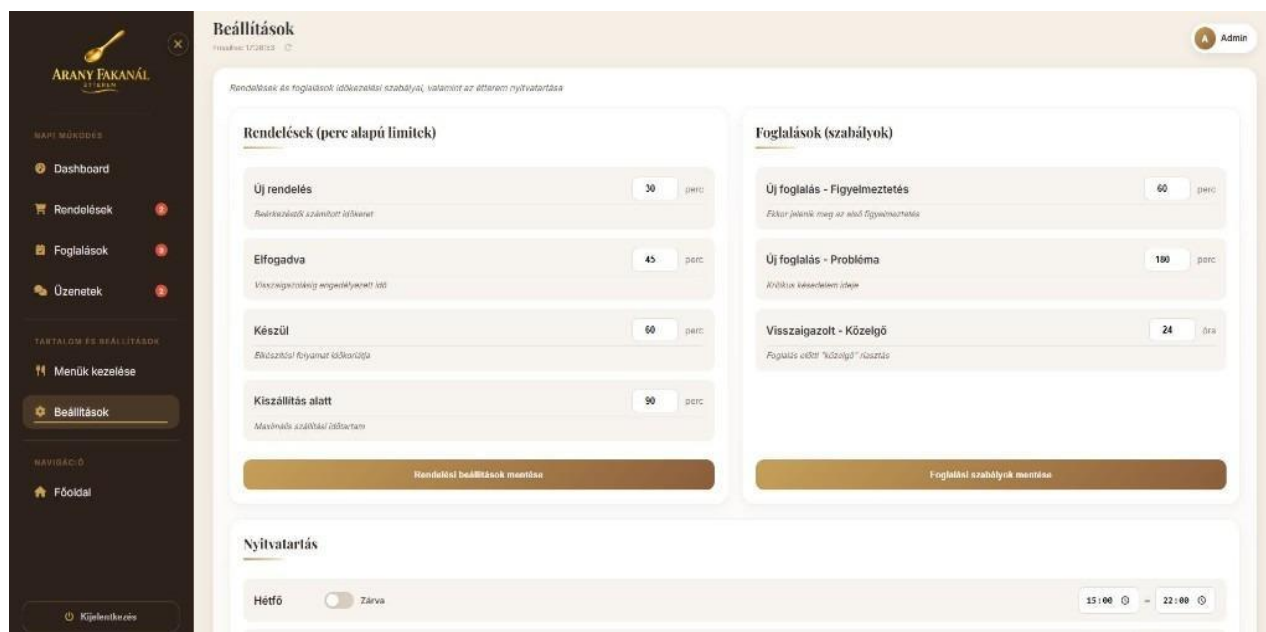
A kapcsolatfelvételi üzenetek külön listában kezelhetők. Az admin itt olvassa el, megválaszolja vagy lezárja a beérkezett üzenetet, élő frissítéssel.

Ábra: Admin dashboard 4/6 — üzenetek

A heti menü A és B ajánlatait az admin ezen a felületen állítja össze napokra bontva. Mentés után a publikus főoldal a live-syncen keresztül frissül, külön közzététel nélkül.

Ábra: Admin dashboard 5/6 — heti menü szerkesztése

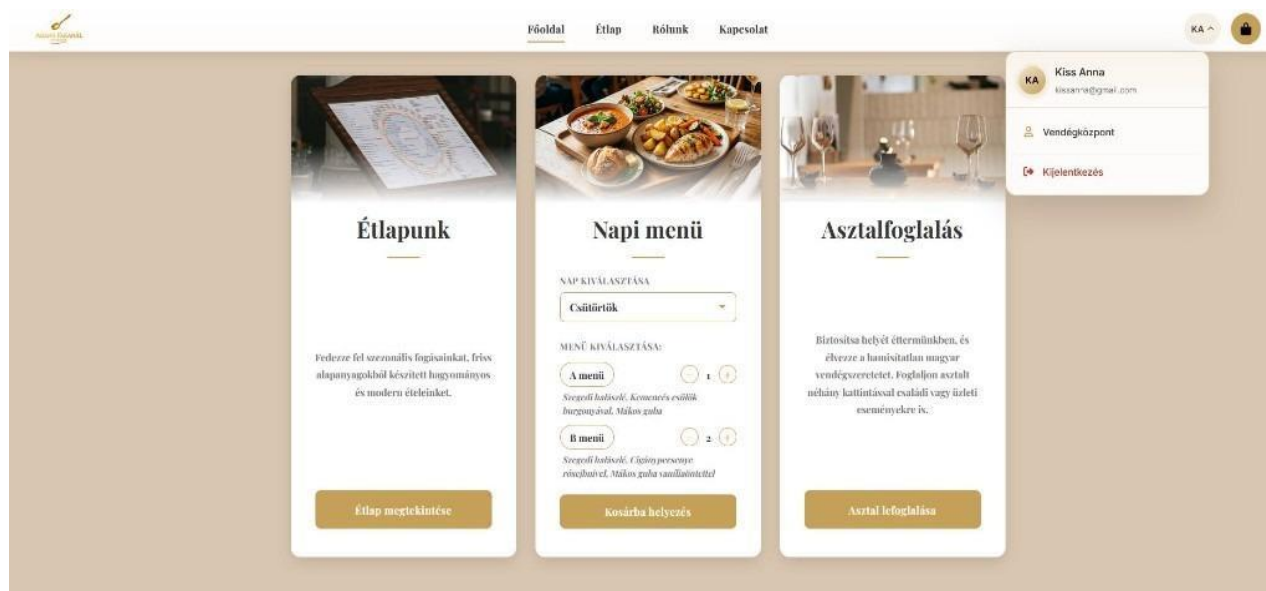
A beállításokban az SLA-szabályok és a nyitvatartás szerkeszthető. Ezek határozzák meg, mikor foglalható asztal, és mikor jelez a dashboard, ha egy tétel túl sokáig kezeletlen.



Ábra: Admin dashboard 6/6 — beállítások (SLA és nyitvatartás)

Vendégközpont

A bejelentkezett vendég a jobb felső profilmenüből nyitja a vendégközpontot. Itt a saját adatait, rendeléseit és foglalásait éri el, külön admin felület nélkül.



Ábra: Vendégközpont 1/4 — regisztrált felhasználó a saját menüjéből léphet be a vendégközpontba

Az Adataim lapon a vendég a nevét, telefonját és a szállítási címet módosíthatja. Az e-mail a belépéshez kötött, ezért itt nem szerkeszthető.

Adataim
Profilod és szállítási adataid kezelése.

Fiók létrehozva: 2026. július 4.
Itt szerkesztheted a rendeléshez és kiszállításához használt adataidat.

Személyes adatok

Teljes név *
Kiss Anna

Telefonszám *
0620123654

Szállítási cím

Szállítási cím *
4024 Debrecen, Thomas Mann utca 23

Biztonság kedvéért a kiszállításnál használjuk ezt a címet.

Bejelentkezés

E-mail cím nem módosítható
kissanna@gmail.com

Az e-mail cím nem módosítható.

[Adataim módosítása](#)

Ábra: Vendégközpont 2/4 — Adataim

A Rendelésem listán csak a bejelentkezett vendég saját tételei látszanak. Amíg egy rendelés Új státuszú, módosítható vagy lemondható; későbbi státusznál már csak megtekinthető.

Rendelésem
Saját rendeléseket, módosítás és lemondás.

Módosítás és lemondás
A rendelésedet addig tudod szerkeszteni vagy lemondani, amíg az adott szállítási nap státusza **Új**. Ha az admin már átvette (pl. Elfogadva, Készül), az a nap záróra marad — ilyenkor a gombok inaktívak.

Aktív (1) **Lezárt (15)**

Következő: Csütörtök, 2026. aug. 6. 4500 Ft

Lezárva: 2026.08.05 17:12

Szállítási cím
4024 Debrecen, Thomas Mann utca 23 [Cím módosítása](#)

Csak ehhez a rendeléshez. Használat, ha időrendileg máshova kéred a kiszállítást, vagy jelenleg szeretnéd a leadáskor megadott címet — az **Adataim** profilod és a főoldal rendelései nem változik.

Kiszállítási napok 2 kiszállítási nap

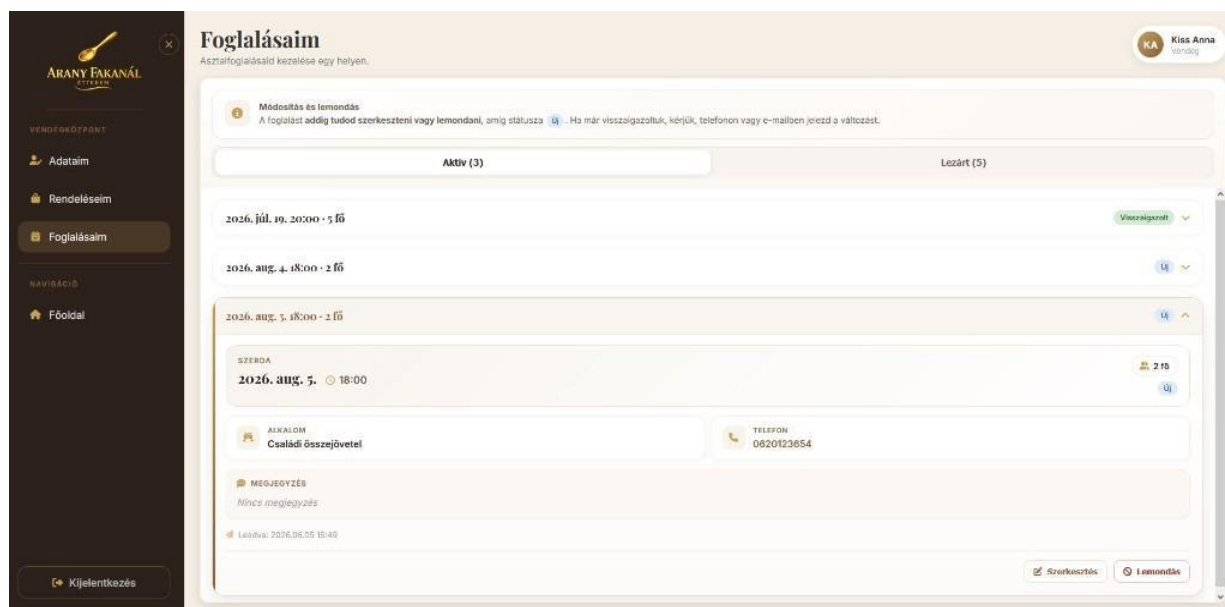
A menü
Leves Szegedi halászlé
Féltel Komocós csülök burgonyával
Desszert Mákos guba **x 1**

[Szerkesztés](#) [Lemondás](#)

B menü
Leves Jókai bobóves
Féltel Komocós csülök burgonyával **x 2**

Ábra: Vendégközpont 3/4 — Rendelésem (módosítás / lemondás)

A Foglalásaim listán a vendég a saját asztalfoglalásait látja. Amíg a foglalás Új státuszú, szerkeszthető vagy lemondható; később már csak megtekinthető.



Ábra: Vendégközpont 4/4 — Foglalásaim

7.2. A működési folyamatok bemutatása

A hét fő folyamatot itt részletesebben végigjárjuk: mit csinál a felhasználó a felületen, mi történik a szerveren, és hol van ehhez a kód a projektben.

1. Regisztráció és bejelentkezés

A nyilvános oldalakon a navbar profilgombja nyitja a bejelentkező és regisztrációs panelt. Regisztrációnál a vendég megadja a nevét, e-mailjét, telefonját, szállítási címét és a jelszavát, plusz elfogadja az adatkezelést. A kérés a POST /api/auth/register/ végpontra megy; a szerver létrehoz egy új usert és a hozzá tartozó UserProfile-t customer szerepkörrel.

Bejelentkezéskor e-mail és jelszó kell. A POST /api/auth/login/ után a szerver munkamenetet indít, a böngésző pedig sessionid cookie-t kap. A későbbi kéréseknél a GET /api/auth/me/ megmondja, ki van bejelentkezve. Vendégként a profilmenüben a vendégközpont jelenik meg; adminként a dashboard. Kijelentkezéskor a POST /api/auth/logout/ törli a sessiont.

Ehhez a backend oldalon a users app view-jai tartoznak (LoginView, RegisterView, LogoutView, MeView), a felületen pedig a login és a navbar JavaScript kezeli a panelt és a cookie-s kéréseket.

A bejelentkezés lényege a LoginView-ban így néz ki (users/views.py):

```
class LoginView(APIView):  
    """  
    POST /api/auth/login/  
    Body: { "email": "...", "password": "..."}  
    Sikeres válasz: felhasználó adatai + session cookie.  
    """  
    permission_classes = [AllowAny] # belépés előtt bárki hívhatja  
    def post(self, request):
```

```

serializer = LoginSerializer(data=request.data)
serializer.is_valid(raise_exception=True)
email = serializer.validated_data["email"].lower()
password = serializer.validated_data["password"]
user_obj = User.objects.filter(
    models.Q(username__iexact=email) | models.Q(email__iexact=email)
).first()
if user_obj is None:
    return Response(
        {"detail": "Hibás e-mail cím vagy jelszó."},
        status=status.HTTP_401_UNAUTHORIZED,
    )
user = authenticate(request, username=user_obj.username,
password=password)
if user is None:
    return Response(
        {"detail": "Hibás e-mail cím vagy jelszó."},
        status=status.HTTP_401_UNAUTHORIZED,
    )
login(request, user) # session létrehozása
return Response(user_to_dict(user))

```

A LoginView a POST /api/auth/login/ végpontot szolgálja ki. Az AllowAny azt jelenti, hogy ezt a hívást belépés előtt is bárki megteheti — különben senki nem tudna bejelentkezni. Először a serializer ellenőrzi, hogy megérkezett-e az e-mail és a jelszó. Ezután a szerver e-mail vagy username alapján megkeresi a felhasználót (így az olyan fiókok is működnek, ahol a username nem az e-mail). Ha nincs ilyen user, vagy a jelszó nem stimmel, 401-es hibát ad („Hibás e-mail cím vagy jelszó”). Ha a jelszó jó, a login(request, user) létrehozza a munkamenetet: ettől kap a böngésző sessionid cookie-t, és a későbbi kéréseknél a szerver tudja, ki van belépve. Válaszként a felhasználó adatait adja vissza JSON-ben.

2. Heti menü rendelés

A főoldalon jelenik meg az aktuális heti menü. A vendég kiválaszt egy napot és A vagy B menüt, majd a kosárba teszi. A kosár a böngésző localStorage-ában van, felhasználónként külön, hogy ne keveredjenek a tételek. Kosárba rakás és leadás csak bejelentkezés után megy: ha valaki nincs belépve, a rendszer előbb belépésre kéri, aztán folytatja.

Házhoz csak munkanapra (hétfő–péntek) lehet rendelni; szombatra és vasárnapra nincs kiszállítás. A választható napok attól függenek, milyen nap van ma. Hétfőn a kedd–péntek napok jelennek meg, kedden a szerda–péntek, szerdán a csütörtök–péntek, csütörtökön csak a péntek. Pénteken, szombaton és vasárnap már a következő teljes hét hétfő–péntek napjaira lehet rendelni. Tehát az aznapi menüt nem lehet „mára” kérni: legkorábban a következő munkanapra szól a kosár.

Leadáskor a frontend a POST /api/orders/create/ címre küldi a rendelést; a szállítási cím a profilból jön. Ha minden rendben van, a kosár kiürül, az admin pedig a dashboardon látja az új rendelést, és állíthatja a státuszát (például Új, Elfogadva, Készül).

A kosár és a heti menü a `frontend/static/js/cart.js` és `weekly-menu.js` fájlokban van (a választható napokat a `getAvailableOrderDays()` számolja); a mentés az `orders` appban történik (`Order` és `OrderItem` modellek).

3. Asztalfoglalás

Az `/asztalfoglalas/` oldalon bárki kitöltheti a foglaló űrlapot — bejelentkezés nem kötelező. Meg kell adni a dátumot, időpontot, létszámot és az alkalmat, valamint a vendég adatait. Ha a felhasználó be van jelentkezve, a név, e-mail és telefon előtöltődik (ezt a `booking-form-prefill.js` intézi).

Mentéskor a szerver megnézi, hogy az adott nap és óra belefér-e a nyitvatartásba, és nincs-e ütközés. Ha igen, a foglalás `pending` (Új) státusszal kerül az adatbázisba. Az admin a `dashboard` Foglalások részén visszaigazolhatja vagy módosíthatja; a vendég a vendégközpontban látja a sajátját, és amíg `pending` (függőben van), le is mondhatja.

A logika a `reservations` appban van, a választható időpontokhoz az `opening_hours` `services` és az `opening-hours.js` tartozik.

4. Kapcsolatüzenet

A főoldal kapcsolat szekciójában a vendég üzenetet írhat az étteremnek. Ehhez nem kell belépni; ha mégis be van jelentkezve, a név és az e-mail előjön. Az adatkezelési pipa után az üzenet elmentődik a `contact` API-n keresztül. Az admin a `dashboard` Üzenetek részén olvassa, jelölheti olvasottnak vagy megválaszoltnak, és archiválhatja.

Frontend oldalon a `contact-form.js`, a dashboardon a `messages.js`, a backend pedig a `contact` app.

5. Admin kezelés (dashboard)

A `/dashboard/` oldalra csak admin szerepkörű felhasználó léphet be. Egy oldalon belül szekciók között vált: áttekintő, rendelések, foglalások, menü, üzenetek, beállítások.

Az áttekintőn látszanak a fontos számok, a teendők és az SLA figyelmeztetések. A rendeléseknél és foglalásoknál listázni, státuszt váltani, szerkeszteni és törölni lehet. Státuszváltáskor a kód a szokásos mentés helyett gyakran közvetlenül az adatbázisban ír, ezért a Django nem küld `signal`-t, és a `revision` számot kézzel kell növelni — különben a többi böngésző nem frissülne. A menü részen a heti menü és az állandó étlap szerkeszthető; a Beállításokban a nyitvatartás és az SLA az `/api/opening-hours/` és `/api/sla-rules/` végpontokon mentődik.

A felület a `frontend/static/dashboard/js/` alatt van (például `index.js`, `orders.js`, `bookings.js`, `settings.js`); a backend az `orders`, `reservations`, `menu`, `contact` és `opening_hours` API-kat szolgálja ki.

6. Vendégközpont

A `/guest-portal/` a bejelentkezett vendég saját oldala. Itt három rész van: Adataim, Rendeléseim és Foglalásaim. Az adatoknál a név, telefon és cím a `PATCH /api/auth/me/` végponton menthető. A rendeléseknél és foglalásoknál aktív és lezárt lista van; módosítani vagy lemondani csak akkor lehet, ha a szerver a `can_edit` / `can_cancel` mezőkben engedi — tipikusan amíg a tétel Új, illetve a foglalás `pending`.

A `guest_portal` appnak nincs saját táblája: az `orders` és `reservations` modelleket szűri a bejelentkezett felhasználóra. A frontend a `frontend/static/guest-portal/js/` mappában van.

7. Élő frissítés

Ha az admin elfogad egy rendelést, a másik gépen vagy böngészőben a lista magától frissül, anélkül hogy bárki F5-öt nyomna. Mentés után nő az AppRevision számláló; a live-sync.js kb. két másodpercenként lekéri a GET /api/revision/ választ. Ha a szám nagyobb, mint amit utoljára látott, újratölti, ami az adott oldalon kell (rendelések, foglalások, menü, üzenetek, nyitvatartás, vendégközpont).

A számlálót a sync app növeli: szokásos mentésnél automatikusan (signal), státuszváltásnál viszont gyakran kézzel, mert ott a kód a szokásos mentés helyett közvetlenül az adatbázisban ír (update), és emiatt a Django nem küld signal-t. A böngésző oldali logika a frontend/static/js/live-sync.js fájlban van.

A live-sync.js az oldal betöltésekor elindul: ha a HTML még töltődik, a DOMContentLoaded eseményre vár, különben azonnal meghívja a start() függvényt. A start() elindít egy időzítőt (setInterval), így a böngésző folyamatosan, kb. két másodpercenként ellenőrzi a revision számot — amíg az oldal nyitva van (háttérben lévő lapon a poll szünetel, hogy ne terhelje feleslegesen a szervert).

A böngésző oldali ellenőrzés lényege a live-sync.js-ben így néz ki (egyszerűsítve):

```
const POLL_MS = 2000;
let revision = null;

async function fetchRevision() {
  const response = await fetch("/api/revision/");
  const data = await response.json();
  return data.revision;
}

async function poll() {
  const current = await fetchRevision();
  if (revision === null) {
    revision = current;
    return;
  }
  if (current !== revision) {
    revision = current;
    await onRevisionChanged(); // listák újratöltése
  }
}

setInterval(poll, POLL_MS);
```

Tehát a figyelés nem egyszeri: a setInterval miatt a poll() újra és újra lefut, és minden alkalommal lekéri a GET /api/revision/ számot. Ha a szám nőtt, meghívja az onRevisionChanged() függvényt, és újratölti a listákat. A revision értékét a szerver növeli mentéskor (bump_revision / signal); update()-nél — például státuszváltáskor — ezt kézzel kell meghívni.

Az onRevisionChanged() azt dönti el, mit kell újratölteni. A revision csak annyit jelez, hogy változott valami — nem tudja, hogy mi. A függvény ezért sorban megnézi: az adott oldalon létezik-e már a frissítő kód. Ezt típusvizsgálattal teszi: ha például egy név function típusú (typeof ... ===

"function"), vagy egy objektum már betöltődött (`typeof ... !== "undefined"`), akkor azon az oldalon az a rész frissítendő, és meghívja. Ha nincs ilyen, továbblép — így nem frissít olyasmit, ami az aktuális képernyőn nincs is:

```
async function onRevisionChanged() {
  if (typeof OpeningHours !== "undefined") {
    await OpeningHours.fetchOpeningHours({ force: true });
    refreshOpeningHoursDisplays();
  }
  if (typeof window.refreshWeeklyMenu === "function") {
    await window.refreshWeeklyMenu();
  }
  if (typeof window.refreshEtlap === "function") {
    await window.refreshEtlap();
  }
  await refreshDashboardData(); // csak ha a dashboard JS betöltődött
  await refreshGuestPortalData(); // csak a vendégközponton
  if (typeof MenuManager !== "undefined") {
    MenuManager.refresh();
  }
  await refreshSettingsPanel();
}
```

Tehát nem egy központi lista mondja meg, hogy „most a rendelést frissítsd”, hanem az, hogy a böngészőben az adott oldalon egyáltalán elérhető-e a frissítő függvény vagy objektum. Főoldalon jellemzően a heti menü és a nyitvatartás, dashboardon a rendelések és üzenetek, vendégközponton a saját listák. A `refreshDashboardData()` ugyanezt a mintát követi belül is.

8. Tesztelés

A rendszer tesztelése során elsősorban manuális funkcionális teszteket végeztünk. A főbb felhasználói folyamatokat a weboldalon keresztül ellenőriztük, többek között a regisztrációt és bejelentkezést, a rendelések leadását és módosítását, az asztalfoglalást, valamint a dashboard funkcióit. A sikeres műveletek mellett a hibás vagy hiányos adatok kezelését is vizsgáltuk: ellenőriztük például az e-mail-címek formátumát, a kötelező mezők kitöltését, az adatkezelési tájékoztató elfogadását, valamint azt, hogy múltbeli időpontra ne lehessen foglalást létrehozni. A frontend megjelenítését nagyobb mennyiségű, tesztelési célból generált adattal is vizsgáltuk. A fejlesztés során azonosított kisebb működési és megjelenítési problémákat a szükséges módosítások után ismételt teszteléssel ellenőriztük.

9. Összegzés

A projekt során egy olyan éttermi rendelési és asztalfoglalási rendszer megvalósítását tűztük ki célul, amely a vendégek és az étterem munkatársainak oldaláról is támogatja a mindennapi működéshez szükséges feladatokat. A fejlesztés során kialakítottuk a rendszer adatmodelljét,

megvalósítottuk a backend és frontend komponenseket, valamint létrehoztuk a rendeléshez, asztalfoglaláshoz, étlapkezeléshez és adminisztrációhoz szükséges funkciókat.

A fejlesztés során az elméleti ismeretek mellett a gyakorlatban is megtapasztaltuk egy összetettebb szoftveres rendszer megtervezésének és megvalósításának folyamatát. A munka során fontos szerepet kapott a feladatok felosztása, a különböző fejlesztési területek összehangolása, a verziókövetés, valamint a folyamatos ellenőrzés és javítás. A projekt során felmerülő problémák megoldása hozzájárult ahhoz, hogy a kezdeti elképzeléseket a tényleges működéshez igazítsuk.

Az elkészült rendszer a projektben meghatározott alapvető célokat teljesíti, ugyanakkor a kialakított struktúra további fejlesztésekre is lehetőséget biztosít. A későbbiekben új funkciókkal, további automatizált folyamatokkal és az éttermi működést támogató megoldásokkal bővíthető. A projekt így nemcsak egy működő alkalmazás létrehozását jelentette, hanem a tervezés, fejlesztés, együttműködés és tesztelés során megszerzett gyakorlati tapasztalatok összegzését is.